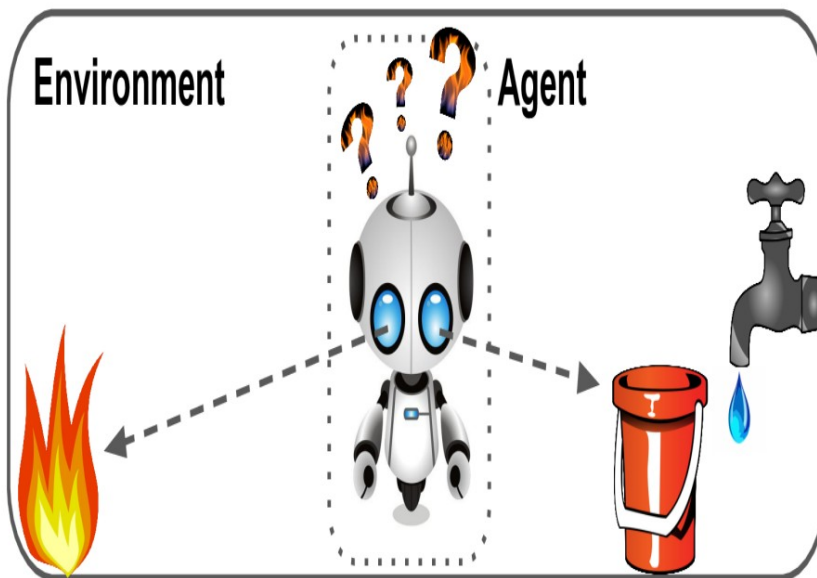
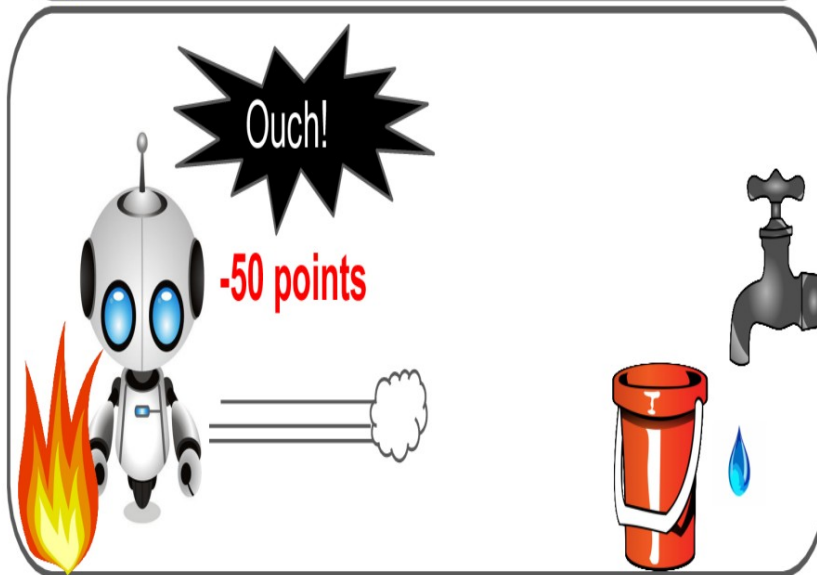




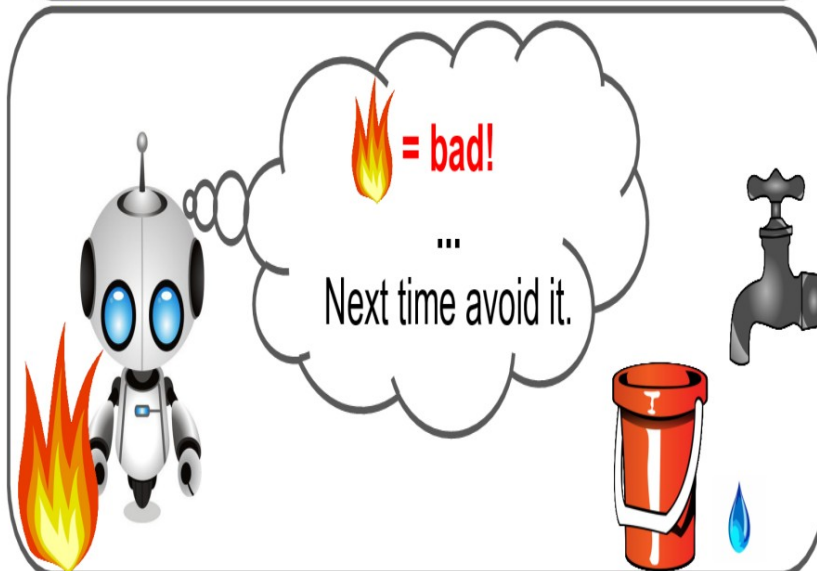
1 Observe

2 Select action using policy



3 Action!

4 Get reward or penalty



5 Update policy (learning step)

6 Iterate until an optimal policy is found

Hình 1-13. Học tăng cường

Ví dụ, nhiều robot sử dụng thuật toán Học tăng cường (Reinforcement Learning) để học cách đi bộ. Chương trình AlphaGo của DeepMind cũng là một ví dụ điển hình về Học tăng cường: nó đã gây chú ý vào tháng 5 năm 2017 khi đánh bại Ke Jie, kỳ thủ số một thế giới vào thời điểm đó, trong trò chơi cờ vây. Nó đã học được chiến lược chiến thắng bằng cách phân tích hàng triệu ván cờ, và sau đó tự chơi nhiều ván cờ với chính mình. Lưu ý rằng quá trình học đã bị tắt trong các ván đấu với nhà vô địch; AlphaGo chỉ áp dụng chiến lược mà nó đã học được. Như chúng ta sẽ thấy trong phần tiếp theo, điều này được gọi là học ngoại tuyến (offline learning).

Học theo nhóm so với học trực tuyến

Một tiêu chí khác được sử dụng để phân loại các hệ thống Học máy là liệu hệ thống đó có thể học hỏi một cách tăng dần từ luồng dữ liệu đầu vào hay không.

Học theo lô

Trong học theo lô, hệ thống không có khả năng học tăng dần: nó phải được huấn luyện bằng cách sử dụng tất cả dữ liệu có sẵn. Điều này thường tốn rất nhiều thời gian và tài nguyên tính toán, vì vậy nó thường được thực hiện ngoại tuyến. Đầu tiên, hệ thống được huấn luyện, sau đó được đưa vào sản xuất và chạy mà không học thêm nữa; nó chỉ áp dụng những gì đã học. Điều này được gọi là học ngoại tuyến.

Thật không may, hiệu suất của một mô hình thường giảm dần theo thời gian, đơn giản vì thế giới tiếp tục phát triển trong khi mô hình vẫn không thay đổi.

Hiện tượng này thường được gọi là sự suy giảm hiệu suất của mô hình hoặc sự thay đổi dữ liệu. Giải pháp là thường xuyên huấn luyện lại mô hình trên dữ liệu cập nhật. Tần suất cần thực hiện việc này phụ thuộc vào trường hợp sử dụng: nếu mô hình phân loại hình ảnh mèo và chó, hiệu suất của nó sẽ giảm rất chậm, nhưng nếu mô hình xử lý các hệ thống phát triển nhanh, ví dụ như đưa ra dự đoán trên thị trường tài chính, thì hiệu suất của nó có thể giảm khá nhanh.

CẢNH BÁO

Ngay cả một mô hình được huấn luyện để phân loại hình ảnh mèo và chó cũng có thể cần được huấn luyện lại thường xuyên, không phải vì mèo và chó sẽ đột biến chỉ sau một đêm, mà vì máy ảnh liên tục thay đổi, cùng với định dạng hình ảnh, độ sắc nét, độ sáng và tỷ lệ kích thước. Hơn nữa, mọi người có thể yêu thích những giống chó khác nhau vào năm tới, hoặc họ có thể quyết định đội những chiếc mũ nhỏ xíu cho thú cưng của mình—ai mà biết được?

Nếu bạn muốn một hệ thống học theo lô biết về dữ liệu mới (chẳng hạn như một loại thư rác mới), bạn cần huấn luyện một phiên bản mới của hệ thống từ đầu trên toàn bộ tập dữ liệu (không chỉ dữ liệu mới mà cả dữ liệu cũ), sau đó thay thế mô hình cũ bằng mô hình mới. May mắn thay, toàn bộ quá trình huấn luyện, đánh giá và triển khai một hệ thống Học máy có thể được tự động hóa khá dễ dàng (như chúng ta đã thấy trong [Hình 1-3](#)), vì vậy ngay cả một hệ thống học theo lô cũng có thể thích ứng với sự thay đổi. Chỉ cần cập nhật dữ liệu và huấn luyện một phiên bản mới của hệ thống từ đầu thường xuyên khi cần thiết.

Giải pháp này đơn giản và thường hoạt động tốt, nhưng việc huấn luyện bằng toàn bộ tập dữ liệu có thể mất nhiều giờ, vì vậy thông thường bạn chỉ nên huấn luyện một hệ thống mới sau mỗi 24 giờ hoặc thậm chí chỉ mỗi tuần một lần. Nếu hệ thống của bạn cần thích ứng với dữ liệu thay đổi nhanh chóng (ví dụ: để dự đoán giá cổ phiếu), thì bạn cần một giải pháp phản ứng nhanh hơn.

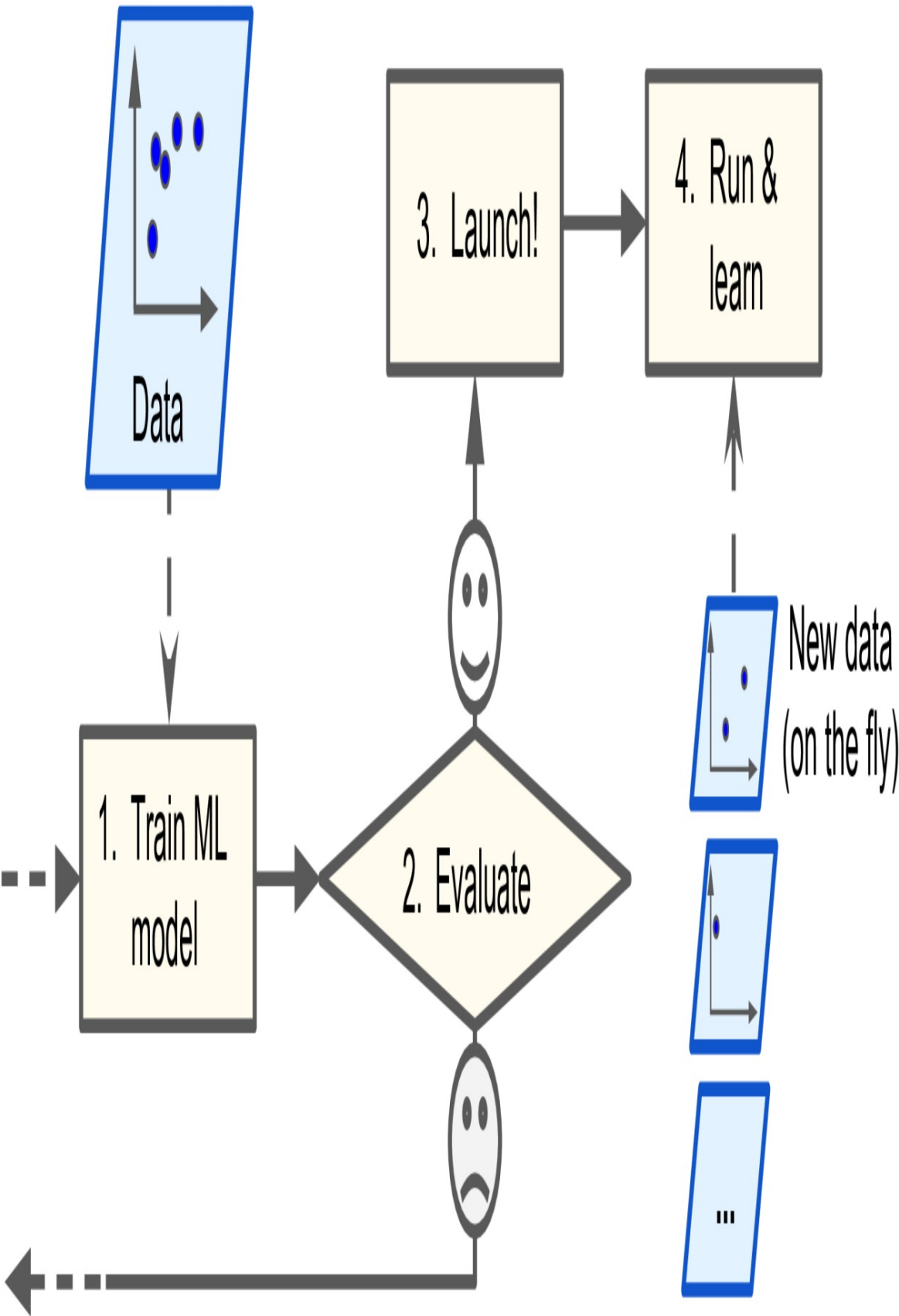
Ngoài ra, việc huấn luyện trên toàn bộ tập dữ liệu đòi hỏi rất nhiều tài nguyên tính toán (CPU, dung lượng bộ nhớ, dung lượng ổ đĩa, I/O ổ đĩa, I/O mạng, v.v.). Nếu bạn có nhiều dữ liệu và tự động hóa hệ thống để huấn luyện lại từ đầu mỗi ngày, cuối cùng sẽ tốn rất nhiều tiền. Nếu lượng dữ liệu quá lớn, thậm chí có thể không thể sử dụng thuật toán học theo lô.

Cuối cùng, nếu hệ thống của bạn cần có khả năng tự học và có nguồn lực hạn chế (ví dụ: ứng dụng trên điện thoại thông minh hoặc robot thám hiểm trên sao Hỏa), thì việc mang theo lượng lớn dữ liệu huấn luyện và tiêu tốn nhiều tài nguyên để huấn luyện hàng giờ mỗi ngày sẽ là một trở ngại lớn.

May mắn thay, trong tất cả các trường hợp này, lựa chọn tốt hơn là sử dụng các thuật toán có khả năng học hỏi một cách tăng dần.

Học trực tuyến

Trong học trực tuyến, bạn huấn luyện hệ thống một cách tăng dần bằng cách cung cấp cho nó các trường hợp dữ liệu theo trình tự, riêng lẻ hoặc theo nhóm nhỏ gọi là mini-batch. Mỗi bước học đều nhanh và tiết kiệm chi phí, do đó hệ thống có thể học về dữ liệu mới ngay lập tức khi dữ liệu được cung cấp (xem **Hình 1-14**).



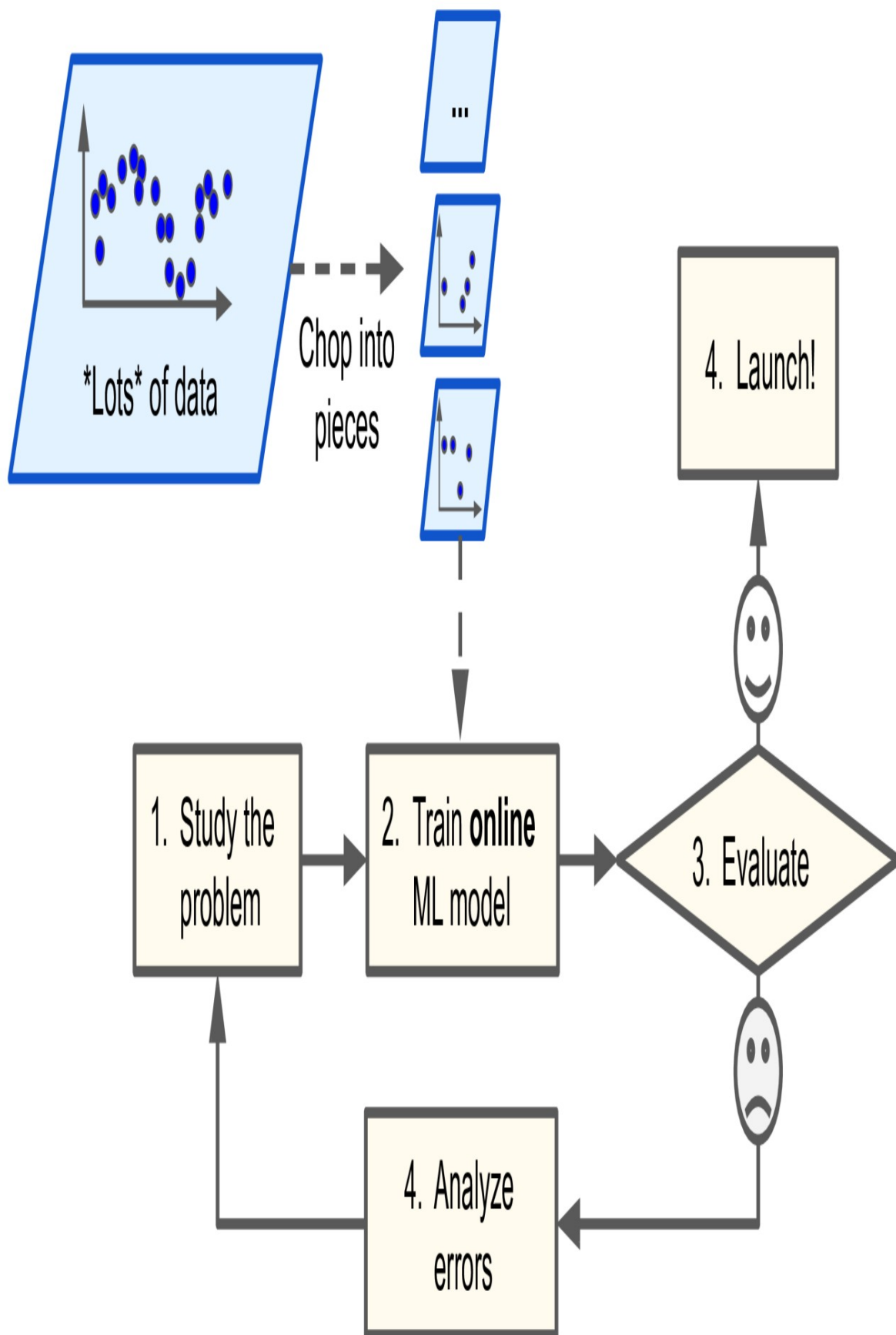
Hình 1-14. Trong học trực tuyến, một mô hình được huấn luyện và đưa vào sử dụng thực tế, sau đó nó tiếp tục học hỏi khi có dữ liệu mới.

Học trực tuyến rất hữu ích cho các hệ thống cần thích ứng với sự thay đổi cực kỳ nhanh chóng (ví dụ: để phát hiện các mô hình mới trên thị trường chứng khoán). Đây cũng là một lựa chọn tốt nếu bạn có nguồn lực tính toán hạn chế, chẳng hạn như khi mô hình được huấn luyện trên thiết bị di động.

Các thuật toán học trực tuyến cũng có thể được sử dụng để huấn luyện các mô hình trên các tập dữ liệu khổng lồ mà không thể chứa vừa trong bộ nhớ chính của một máy (điều này được gọi là học ngoài bộ nhớ). Thuật toán tải một phần dữ liệu, chạy bước huấn luyện trên dữ liệu đó và lặp lại quá trình cho đến khi chạy trên toàn bộ dữ liệu (xem [Hình 1-15](#)).

CẢNH BÁO

Việc học ngoài chương trình giảng dạy thường được thực hiện ngoại tuyến (tức là không trên hệ thống trực tuyến), vì vậy thuật ngữ "học trực tuyến" có thể gây nhầm lẫn. Hãy coi đó như một hình thức học tập tăng dần.



Hình 1-15. Sử dụng học trực tuyến để xử lý các tập dữ liệu khổng lồ

Một thông số quan trọng của các hệ thống học trực tuyến là tốc độ thích ứng với dữ liệu thay đổi: thông số này được gọi là tốc độ học. Nếu bạn đặt tốc độ học cao, hệ thống sẽ nhanh chóng thích ứng với dữ liệu mới, nhưng cũng sẽ nhanh chóng quên dữ liệu cũ (bạn không muốn bộ lọc thư rác chỉ gắn cờ những loại thư rác mới nhất mà nó được hiển thị). Ngược lại, nếu bạn đặt tốc độ học thấp, hệ thống sẽ có quán tính lớn hơn; nghĩa là, nó sẽ học chậm hơn, nhưng cũng sẽ ít nhạy cảm hơn với nhiễu trong dữ liệu mới hoặc với các chuỗi điểm dữ liệu không đại diện (ngoại lệ).

Một thách thức lớn đối với học trực tuyến là nếu dữ liệu kém chất lượng được đưa vào hệ thống, hiệu suất của hệ thống sẽ giảm sút, có thể rất nhanh (tùy thuộc vào chất lượng dữ liệu và tốc độ học). Nếu đó là một hệ thống đang hoạt động, khách hàng của bạn sẽ nhận thấy điều đó. Ví dụ, dữ liệu xấu có thể đến từ lỗi phần mềm (ví dụ: cảm biến bị trục trặc trên robot), hoặc có thể đến từ ai đó đang cố gắng gian lận hệ thống (ví dụ: gửi thư rác vào công cụ tìm kiếm để cố gắng đạt thứ hạng cao trong kết quả tìm kiếm). Để giảm thiểu rủi ro này, bạn cần giám sát hệ thống chặt chẽ và nhanh chóng tắt quá trình học tập (và có thể khôi phục lại trạng thái hoạt động trước đó) nếu phát hiện hiệu suất giảm sút. Bạn cũng có thể muốn giám sát dữ liệu đầu vào và phản ứng với dữ liệu bất thường, ví dụ như sử dụng thuật toán phát hiện bất thường (xem [Chương 9](#)).

Học dựa trên trường hợp so với học dựa trên mô hình

Một cách khác để phân loại các hệ thống Học máy là dựa trên khả năng khái quát hóa của chúng. Hầu hết các nhiệm vụ Học máy đều liên quan đến việc đưa ra dự đoán.

Điều này có nghĩa là, với một số lượng ví dụ huấn luyện nhất định, hệ thống cần có khả năng đưa ra dự đoán tốt (khái quát hóa) cho các ví dụ mà nó chưa từng thấy trước đây. Có một chỉ số hiệu suất tốt trên dữ liệu huấn luyện là tốt, nhưng chưa đủ; mục tiêu thực sự là hoạt động tốt trên các trường hợp mới.

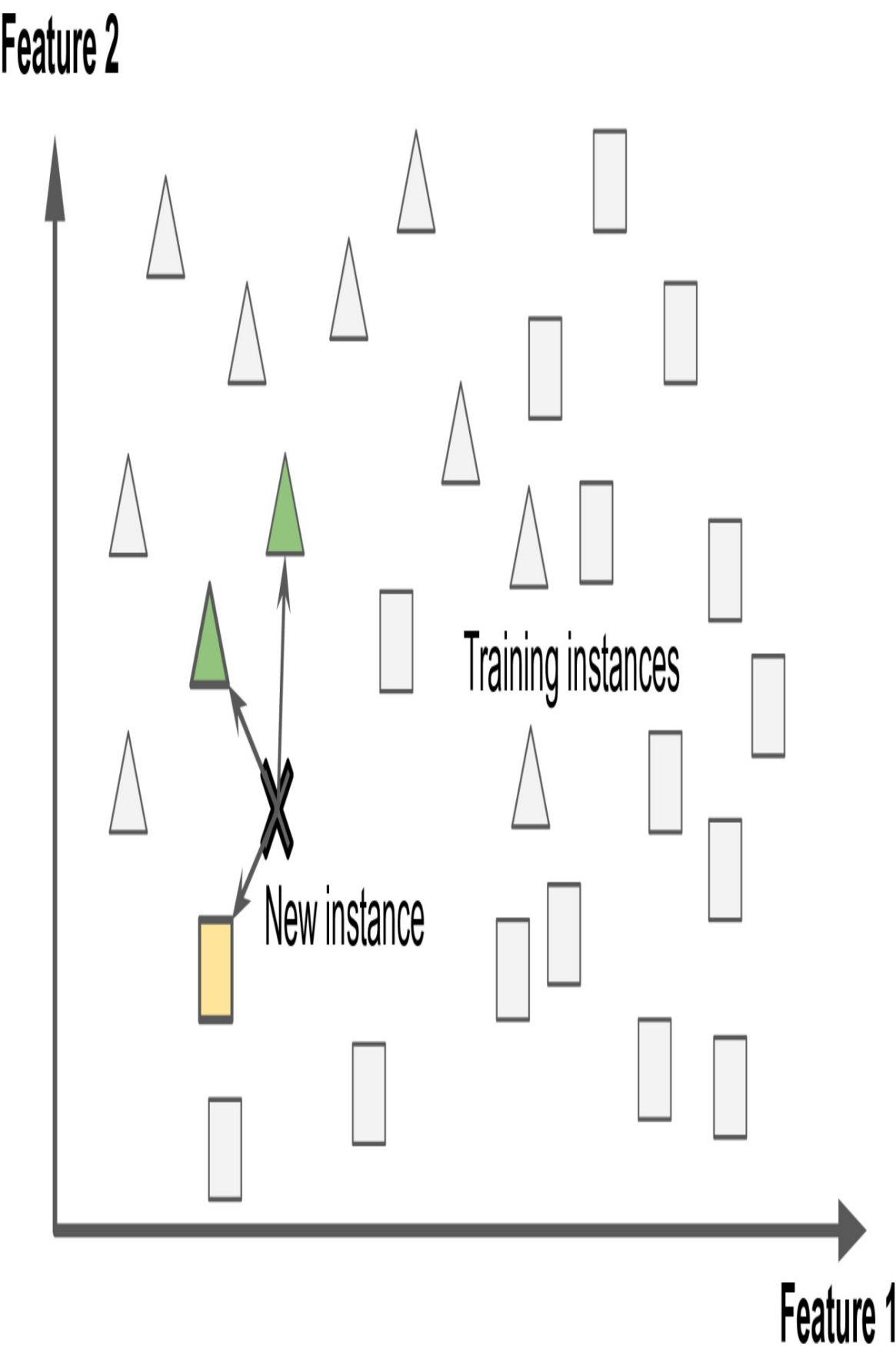
Có hai phương pháp chính để khái quát hóa: học dựa trên trường hợp cụ thể và học dựa trên mô hình.

Học tập dựa trên ví dụ

Có lẽ hình thức học tập đơn giản nhất chính là học thuộc lòng. Nếu bạn tạo bộ lọc thư rác theo cách này, nó sẽ chỉ gắn cờ tất cả các email giống hệt với những email mà người dùng đã gắn cờ trước đó - không phải là giải pháp tồi, nhưng chắc chắn không phải là giải pháp tốt nhất.

Thay vì chỉ gắn cờ những email giống hệt với các email spam đã biết, bộ lọc spam của bạn có thể được lập trình để gắn cờ cả những email rất giống với các email spam đã biết. Điều này đòi hỏi một thước đo độ tương đồng giữa hai email. Một thước đo độ tương đồng (rất cơ bản) giữa hai email có thể là đếm số từ mà chúng có chung. Hệ thống sẽ gắn cờ một email là spam nếu nó có nhiều từ chung với một email spam đã biết.

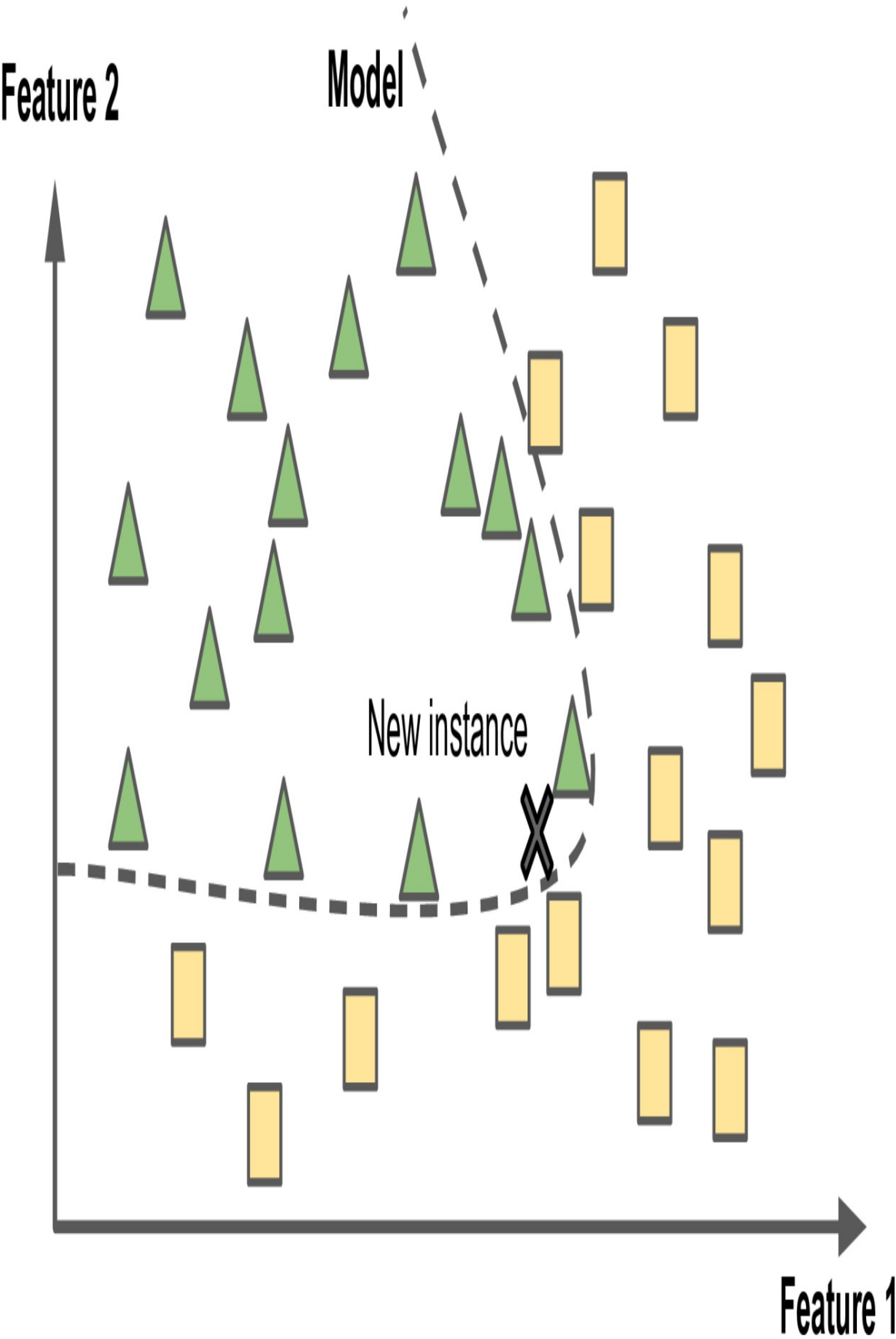
Đây được gọi là học dựa trên ví dụ: hệ thống học thuộc lòng các ví dụ, sau đó khái quát hóa cho các trường hợp mới bằng cách sử dụng thước đo độ tương đồng để so sánh chúng với các ví dụ đã học (hoặc một tập hợp con của chúng). Ví dụ, trong **Hình 1-16**, trường hợp mới sẽ được phân loại là hình tam giác vì phần lớn các trường hợp tương tự nhất thuộc về lớp đó.



Hình 1-16. Học tập dựa trên ví dụ

Học dựa trên mô hình và quy trình làm việc điển hình của Học máy.

Một cách khác để khái quát hóa từ một tập hợp các ví dụ là xây dựng một mô hình từ những ví dụ này và sau đó sử dụng mô hình đó để đưa ra dự đoán. Điều này được gọi là học dựa trên mô hình (Hình 1-17).



Hình 1-17. Học tập dựa trên mô hình

Ví dụ, giả sử bạn muốn biết liệu tiền có làm cho con người hạnh phúc hay không, vì vậy bạn tải xuống dữ liệu Chỉ số Cuộc sống Tốt đẹp hơn từ trang web của OECD và số liệu thống kê của Ngân hàng Thế giới về tổng sản phẩm quốc nội (GDP) bình quân đầu người từ OurWorldInData.org. Sau đó, bạn kết hợp các bảng và sắp xếp theo GDP bình quân đầu người. **Bảng 1-1** trình bày một phần những gì bạn nhận được.

Bảng

1e1-1

•

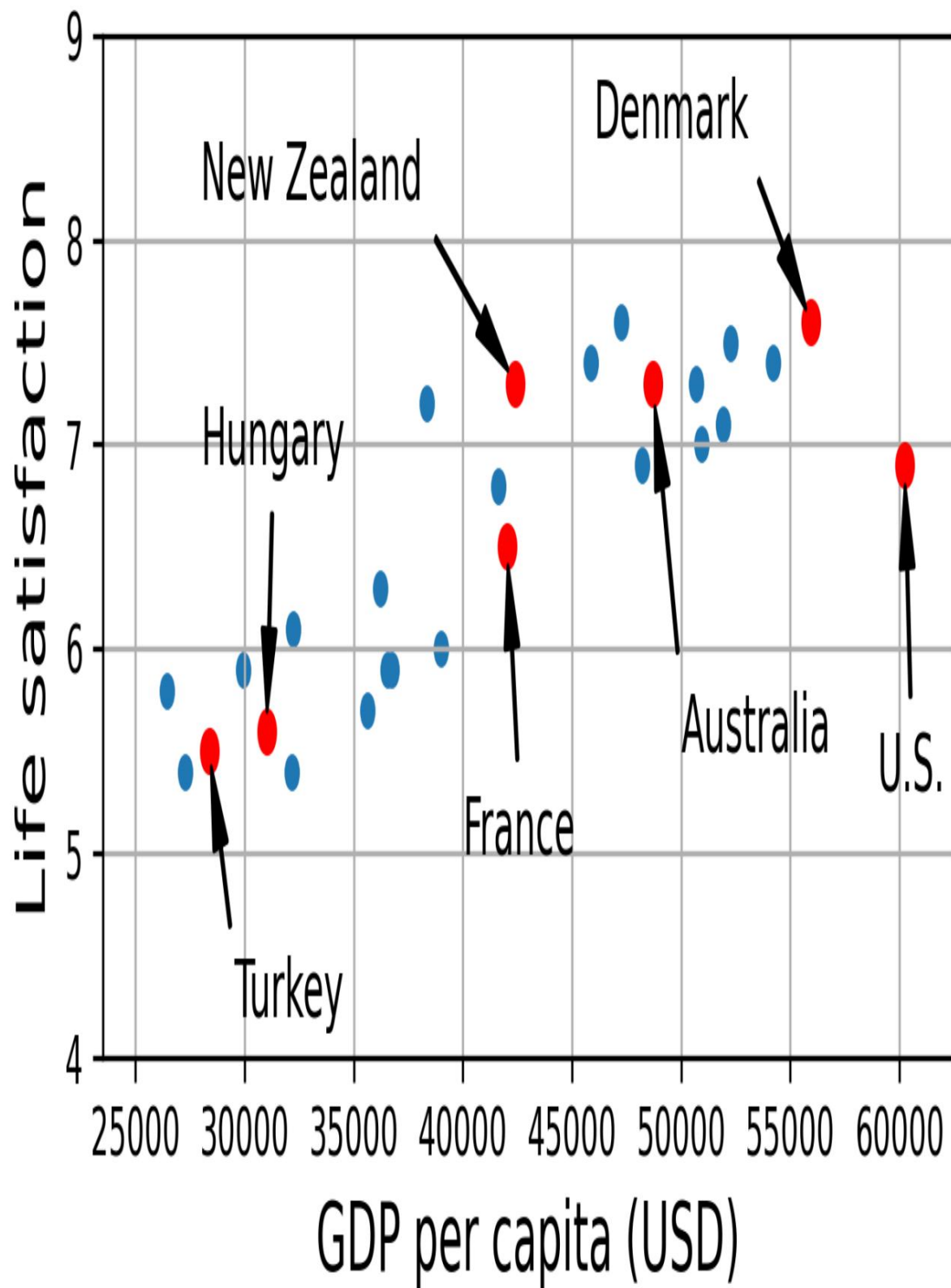
.....

1ehapp

ier?

Quốc gia	GDP bình quân đầu người (USD)	Mức độ hài lòng về cuộc sống
Thổ Nhĩ Kỳ	28.384	5.5
Hungary	31.008	5.6
Pháp	42.026	6,5
Hoa Kỳ	60.236	6.9
New Zealand	42.404	7.3
Úc	48.698	7.3
Đan Mạch	55.938	7.6

Hãy cùng vẽ biểu đồ dữ liệu cho các quốc gia này (Hình 1-18).



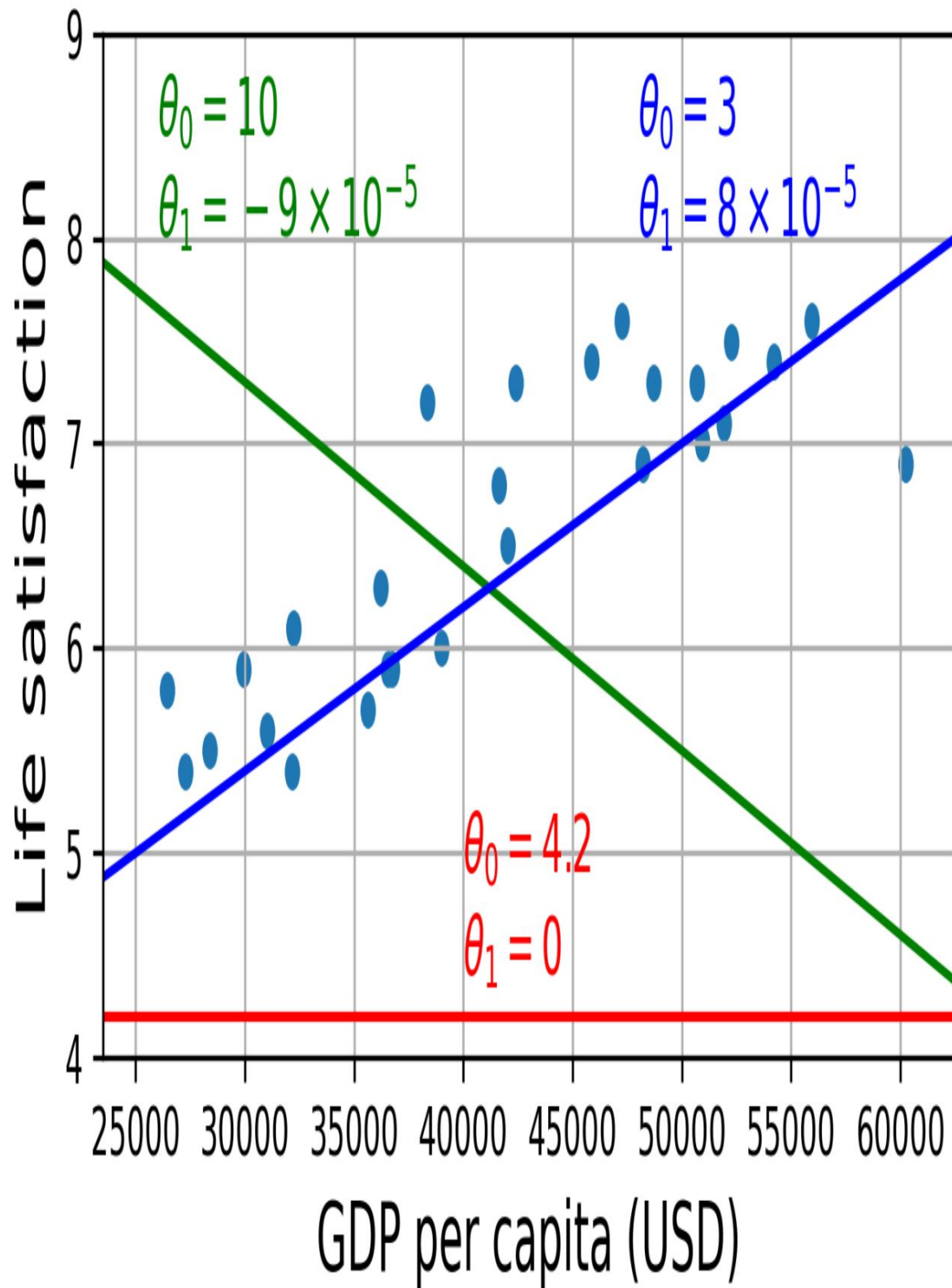
Hình 1-18. Bạn có thấy xu hướng nào ở đây không?

Có vẻ như có một xu hướng ở đây! Mặc dù dữ liệu có nhiều nhiễu (tức là một phần ngẫu nhiên), nhưng dường như sự hài lòng về cuộc sống tăng lên gần như tuyến tính khi GDP bình quân đầu người của quốc gia tăng lên. Vì vậy, bạn quyết định mô hình hóa sự hài lòng về cuộc sống như một hàm tuyến tính của GDP bình quân đầu người. Bước này được gọi là lựa chọn mô hình: bạn đã chọn một mô hình tuyến tính về sự hài lòng về cuộc sống chỉ với một thuộc tính, GDP bình quân đầu người (Phương trình 1-1).

Phương trình 1-1. Một mô hình tuyến tính đơn giản

$$\text{sự hài lòng về cuộc sống} = \theta_0 + \theta_1 \times \text{GDP bình quân đầu người}$$

Mô hình này có hai tham số, tham số θ_0 và θ_1 , bạn có thể làm ⁴ bằng cách điều chỉnh những điều này cho mô hình của mình biểu diễn bất kỳ hàm tuyến tính nào, như được minh họa trong Hình 1-19.



Hình 1-19. Một vài mô hình tuyến tính khả thi

Trước khi có thể sử dụng mô hình, bạn cần xác định các giá trị tham số θ và θ .

0

1. Làm sao bạn có thể biết được những giá trị nào sẽ giúp mô hình của bạn hoạt động tốt nhất? Để trả lời câu hỏi này, bạn cần xác định một thước đo hiệu suất. Bạn có thể định nghĩa một hàm tiện ích (hoặc hàm đánh giá) để đo lường mức độ tốt của mô hình, hoặc bạn có thể định nghĩa một hàm chi phí để đo lường mức độ tệ của mô hình. Đối với các bài toán hồi quy tuyến tính, người ta thường sử dụng hàm chi phí để đo khoảng cách giữa dự đoán của mô hình tuyến tính và các ví dụ huấn luyện; mục tiêu là giảm thiểu khoảng cách này.

Đây là lúc thuật toán Hồi quy tuyến tính phát huy tác dụng: bạn cung cấp cho nó các ví dụ huấn luyện, và nó sẽ tìm ra các tham số làm cho mô hình tuyến tính phù hợp nhất với dữ liệu của bạn. Quá trình này được gọi là huấn luyện mô hình. Trong trường hợp của chúng ta, thuật toán tìm ra rằng các giá trị tham số tối ưu là $\theta = 3,75$ và $\theta = -5,78 \times 10^{-5}$.

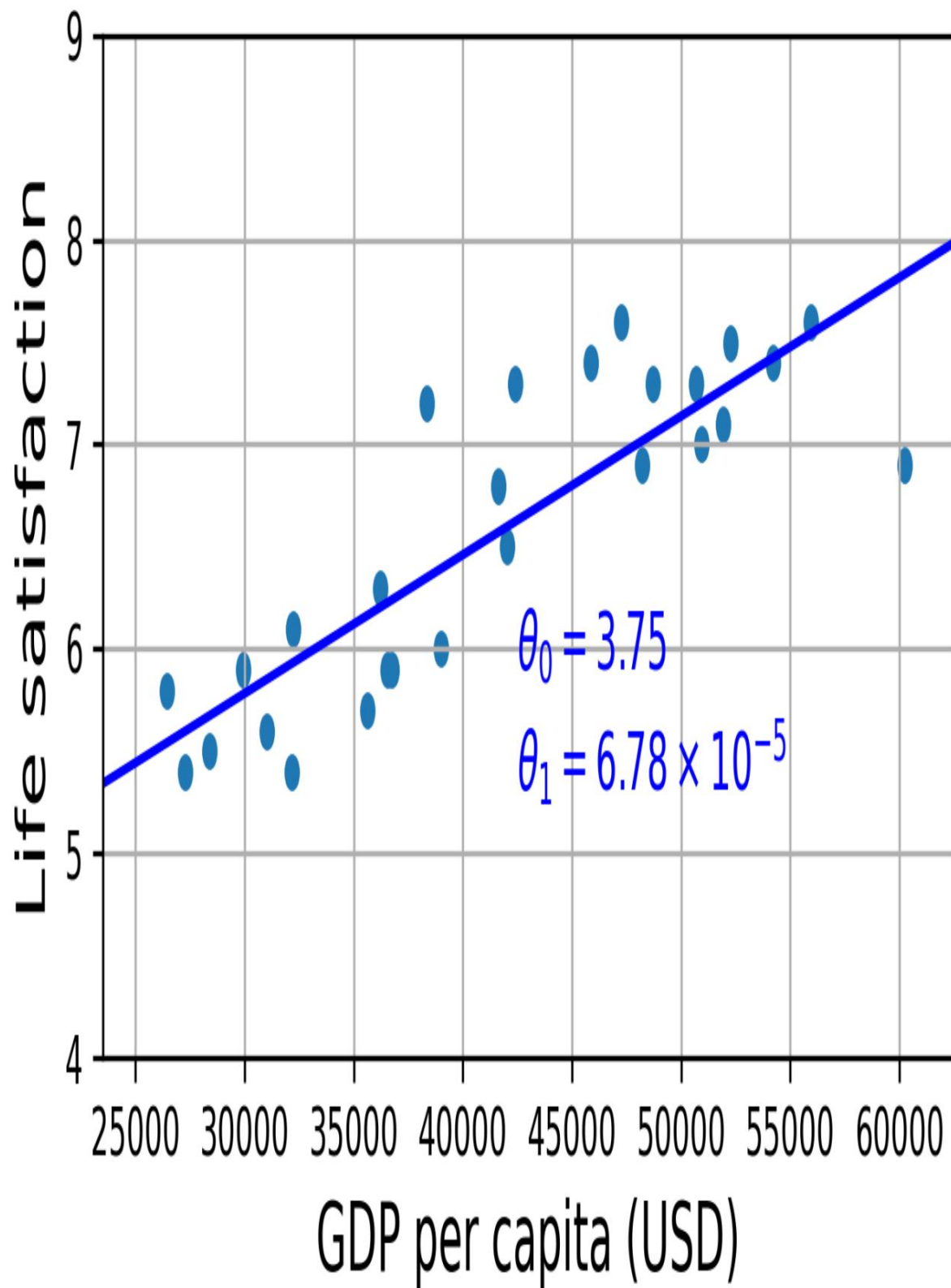
0

1

CẢNH BÁO

Điều gây nhầm lẫn là cùng một từ "mô hình" có thể dùng để chỉ một loại mô hình (ví dụ: Hồi quy tuyến tính), một kiến trúc mô hình được xác định đầy đủ (ví dụ: Hồi quy tuyến tính với một đầu vào và một đầu ra), hoặc mô hình đã được huấn luyện cuối cùng sẵn sàng để sử dụng cho dự đoán (ví dụ: Hồi quy tuyến tính với một đầu vào và một đầu ra, sử dụng $\theta = 3,75$ và $\theta = -5,78 \times 10^{-5}$). Lựa chọn mô hình bao gồm việc chọn loại mô hình và xác định đầy đủ kiến trúc của nó. Huấn luyện mô hình nghĩa là chạy một thuật toán để tìm ra các tham số của mô hình sao cho nó phù hợp nhất với dữ liệu huấn luyện, và hy vọng sẽ đưa ra dự đoán tốt trên dữ liệu mới.

Giờ đây, mô hình khớp với dữ liệu huấn luyện càng sát càng tốt (đối với mô hình tuyến tính), như bạn có thể thấy trong **Hình 1-20**.



Hình 1-20. Mô hình tuyến tính phù hợp nhất với dữ liệu huấn luyện.

Cuối cùng bạn đã sẵn sàng chạy mô hình để đưa ra dự đoán. Ví dụ, giả sử bạn muốn biết người dân Síp hạnh phúc đến mức nào, và dữ liệu của OECD không có câu trả lời. May mắn thay, bạn có thể sử dụng mô hình của mình để đưa ra một dự đoán tốt: bạn tra cứu GDP bình quân đầu người của Síp, thấy con số là 37.655 đô la, sau đó áp dụng mô hình của mình và thấy rằng mức độ hài lòng về cuộc sống có thể nằm trong khoảng $3,75 + 37.655 \times 6,78 \times 10^{-5} = 6,30$.

Để khởi gởi sự tò mò của bạn, Ví dụ 1-1 trình bày mã Python tải dữ liệu, tách các đầu vào X khỏi nhãn y, tạo biểu đồ phân tán cho

Sau đó, hệ thống sẽ trực quan hóa dữ liệu, huấn luyện một mô hình tuyến tính và đưa ra dự đoán.⁵

Ví dụ 1-1. Huấn luyện và chạy mô hình tuyến tính bằng Scikit-Learn [import](#)

```
matplotlib.pyplot as plt import numpy as
np import pandas as pd
from sklearn.linear_model
import LinearRegression

# Tải xuống và chuẩn bị dữ liệu data_root
= "https://github.com/ageron/data/raw/main/" lifesat = pd.read_csv(data_root
+ "lifesat/lifesat.csv")
X = lifesat[["GDP bình quân đầu người (USD)"]].values y =
lifesat[[" Sự hài lòng về cuộc sống"]].values

# Hiển thị dữ liệu
lifesat.plot(kind='scatter', grid=True,
              x="GDP bình quân đầu người (USD)", y="Mức độ hài lòng về cuộc sống")
plt.axis([23_500, 62_500, 4, 9]) plt.show()

# Chọn một mô hình tuyến tính model =
LinearRegression()

# Huấn luyện mô hình
model.fit(X, y)

# Đưa ra dự đoán cho Síp X_new =
[[37_655.2]] # GDP bình quân đầu người của Síp năm 2020
print(model.predict(X_new)) # xuất ra [[6.30165767]]
```

GHI CHÚ

Nếu bạn sử dụng thuật toán học dựa trên trường hợp cụ thể, bạn sẽ thấy rằng GDP bình quân đầu người của Israel gần nhất với Síp (38.341 đô la), và vì dữ liệu của OECD cho thấy mức độ hài lòng về cuộc sống của người Israel là 7,2, bạn sẽ dự đoán mức độ hài lòng về cuộc sống của Síp cũng là 7,2. Nếu bạn nhìn rộng hơn một chút và xem xét hai quốc gia gần nhất tiếp theo, bạn sẽ thấy Litva và Slovenia, cả hai đều có mức độ hài lòng về cuộc sống là 5,9. Bằng cách lấy trung bình ba giá trị này, ta được 6,33, khá gần với dự đoán dựa trên mô hình của bạn. Thuật toán đơn giản này được gọi là hồi quy k-Nearest Neighbors (trong ví dụ này, $k = 3$).

Việc thay thế mô hình Hồi quy tuyến tính bằng mô hình hồi quy k-Hàng xóm gần nhất trong đoạn mã trước đó rất đơn giản, chỉ cần thay thế hai dòng này:

```
from sklearn.linear_model import LinearRegression model = LinearRegression()
```

với hai người này:

```
from sklearn.neighbors import KNeighborsRegressor model =  
KNeighborsRegressor(n_neighbors=3)
```

Nếu mọi việc suôn sẻ, mô hình của bạn sẽ đưa ra những dự đoán tốt. Nếu không, bạn có thể cần sử dụng thêm các thuộc tính (tỷ lệ việc làm, sức khỏe, ô nhiễm không khí, v.v.), thu thập thêm hoặc dữ liệu huấn luyện chất lượng tốt hơn, hoặc có thể chọn một mô hình mạnh hơn (ví dụ: mô hình hồi quy đa thức).

Tóm lại:

- Bạn đã nghiên cứu dữ liệu.
- Bạn đã chọn một mẫu.
- Bạn đã huấn luyện nó trên dữ liệu huấn luyện (tức là thuật toán học máy đã tìm kiếm các giá trị tham số của mô hình sao cho hàm chi phí được tối thiểu hóa).
- Cuối cùng, bạn đã áp dụng mô hình để đưa ra dự đoán về các trường hợp mới (quá trình này được gọi là suy luận), với hy vọng rằng mô hình này sẽ có khả năng khái quát hóa tốt.

Đây là hình ảnh điển hình của một dự án Học máy. Trong **Chương 2**, bạn sẽ được trải nghiệm điều này trực tiếp bằng cách thực hiện một dự án từ đầu đến cuối.

Cho đến nay, chúng ta đã tìm hiểu được rất nhiều nội dung: bạn đã biết Machine Learning thực sự là gì, tại sao nó hữu ích, một số loại hệ thống ML phổ biến nhất là gì và quy trình làm việc điển hình của một dự án trông như thế nào. Bây giờ, hãy cùng xem xét những lỗi có thể xảy ra trong quá trình học và ngăn cản bạn đưa ra dự đoán chính xác.

Những thách thức chính của học máy

Tóm lại, vì nhiệm vụ chính của bạn là chọn một mô hình và huấn luyện nó trên một số dữ liệu, nên hai điều có thể xảy ra sai sót là "mô hình tồi" và "dữ liệu tồi". Hãy bắt đầu với các ví dụ về dữ liệu tồi.

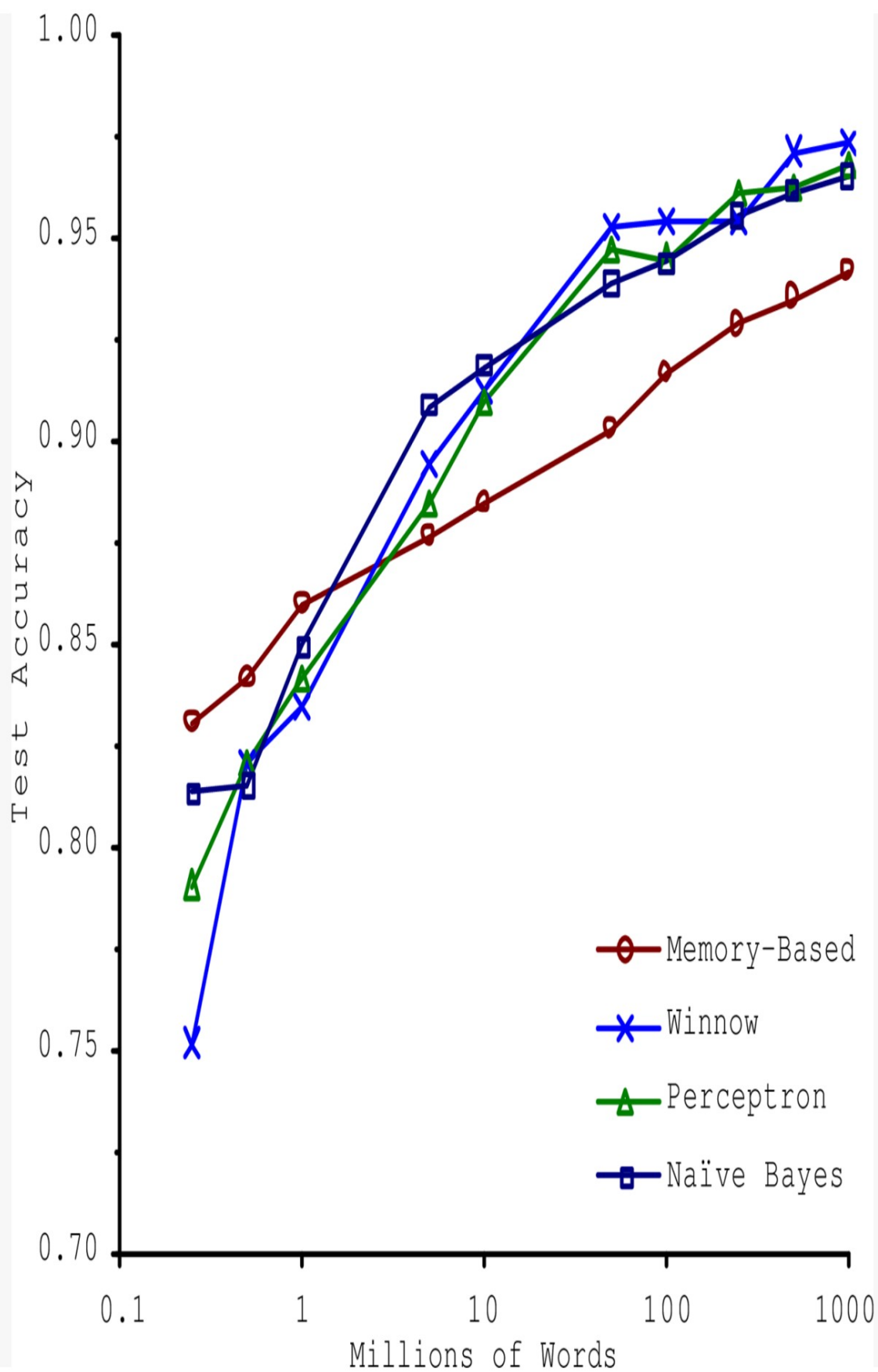
Số lượng dữ liệu huấn luyện không đủ

Để trẻ mới biết đi học được quả táo là gì, tất cả những gì bạn cần làm là chỉ vào một quả táo và nói "táo" (có thể lặp lại thao tác này vài lần). Lúc này, trẻ đã có thể nhận biết được táo với nhiều màu sắc và hình dạng khác nhau. Thiên tài.

Học máy vẫn chưa hoàn thiện; hầu hết các thuật toán học máy cần rất nhiều dữ liệu để hoạt động đúng cách. Ngay cả đối với những bài toán rất đơn giản, bạn thường cần hàng nghìn ví dụ, và đối với những bài toán phức tạp như nhận dạng hình ảnh hoặc giọng nói, bạn có thể cần hàng triệu ví dụ (trừ khi bạn có thể tái sử dụng các phần của mô hình hiện có).

HIỆU QUẢ KHÔNG HỢP LÝ CỦA DỮ LIỆU

Trong một bài báo nổi tiếng được công bố vào năm 2001, các nhà nghiên cứu của Microsoft, Michele Banko và Eric Brill, đã chỉ ra rằng các thuật toán Học máy rất khác nhau, bao gồm cả những thuật toán khá đơn giản, hoạt động gần như tốt như nhau trên một bài toán phức tạp về phân giải ngôn ngữ tự nhiên⁶ khi chúng được cung cấp đủ dữ liệu (như bạn có thể thấy trong Hình 1-21).



Hình 1-21. Tầm quan trọng của dữ liệu so với thuật toán⁷

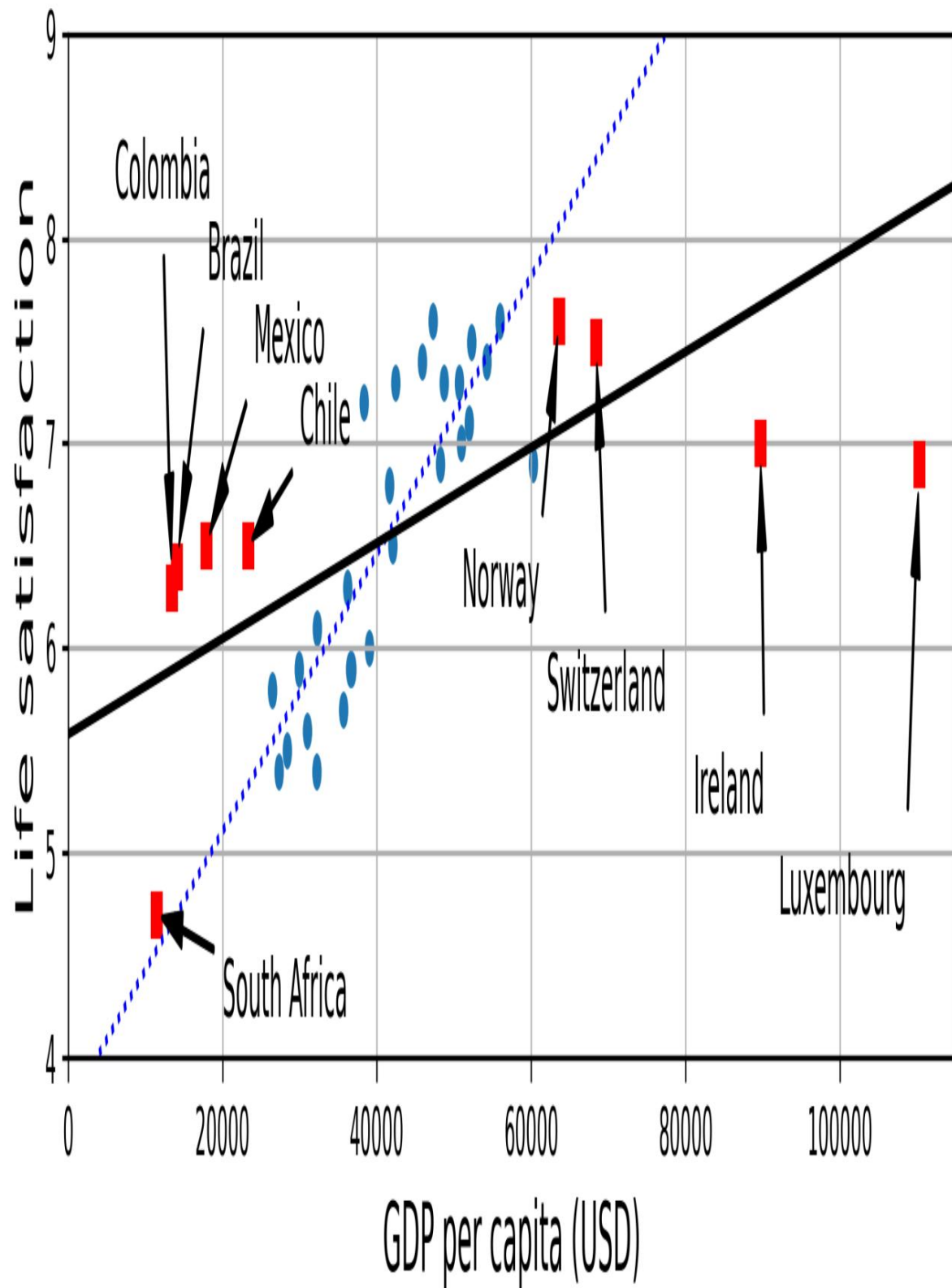
Như các tác giả đã nêu, “những kết quả này cho thấy chúng ta có thể cần xem xét lại sự đánh đổi giữa việc dành thời gian và tiền bạc cho việc phát triển thuật toán so với việc dành chúng cho việc phát triển kho ngữ liệu.”

Quan điểm cho rằng dữ liệu quan trọng hơn thuật toán đối với các vấn đề phức tạp đã được Peter Norvig và cộng sự phổ biến rộng rãi hơn trong bài báo có tựa đề “**Hiệu quả phi lý của dữ liệu**”, xuất bản năm 2009. Tuy nhiên, cần ⁸lưu ý rằng các tập dữ liệu nhỏ và trung bình vẫn rất phổ biến, và việc thu thập thêm dữ liệu huấn luyện không phải lúc nào cũng dễ dàng hoặc rẻ tiền – vì vậy đừng vội từ bỏ thuật toán.

Dữ liệu huấn luyện không mang tính đại diện

Để có khả năng khái quát hóa tốt, điều quan trọng là dữ liệu huấn luyện của bạn phải đại diện cho các trường hợp mới mà bạn muốn khái quát hóa. Điều này đúng cho dù bạn sử dụng học dựa trên trường hợp cụ thể hay học dựa trên mô hình.

Ví dụ, tập hợp các quốc gia mà chúng ta đã sử dụng trước đó để huấn luyện mô hình tuyến tính không hoàn toàn mang tính đại diện; nó không bao gồm bất kỳ quốc gia nào có GDP bình quân đầu người thấp hơn 23.500 đô la hoặc cao hơn 62.500 đô la. **Hình 1-22** cho thấy dữ liệu trông như thế nào khi bạn thêm các quốc gia như vậy.



Hình 1-22. Một mẫu huấn luyện tiêu biểu hơn.

Nếu bạn huấn luyện một mô hình tuyến tính trên dữ liệu này, bạn sẽ nhận được đường liền nét, trong khi mô hình cũ được biểu diễn bằng đường chấm chấm. Như bạn có thể thấy, việc thêm một vài quốc gia bị thiếu không chỉ làm thay đổi đáng kể mô hình mà còn cho thấy rõ ràng rằng một mô hình tuyến tính đơn giản như vậy có lẽ sẽ không bao giờ hoạt động tốt. Dường như các quốc gia rất giàu không hạnh phúc hơn các quốc gia có mức độ giàu vừa phải (thực tế, họ có vẻ kém hạnh phúc hơn một chút!), và ngược lại, một số quốc gia nghèo lại có vẻ hạnh phúc hơn nhiều quốc gia giàu có.

Bằng cách sử dụng tập dữ liệu huấn luyện không đại diện, chúng tôi đã huấn luyện một mô hình khó có thể đưa ra dự đoán chính xác, đặc biệt là đối với các quốc gia rất nghèo và rất giàu.

Việc sử dụng tập dữ liệu huấn luyện mang tính đại diện cho các trường hợp mà bạn muốn khái quát hóa là vô cùng quan trọng. Điều này thường khó hơn tưởng tượng: nếu mẫu quá nhỏ, bạn sẽ gặp phải nhiễu lấy mẫu (tức là dữ liệu không đại diện do ngẫu nhiên), nhưng ngay cả những mẫu rất lớn cũng có thể không đại diện nếu phương pháp lấy mẫu có sai sót. Điều này được gọi là sai lệch lấy mẫu.

CÁC VÍ DỤ VỀ SAI LỆCH TRONG LẤY MẪU

Có lẽ ví dụ nổi tiếng nhất về sai lệch lấy mẫu xảy ra trong cuộc bầu cử tổng thống Mỹ năm 1936, giữa Landon và Roosevelt: tạp chí Literary Digest đã tiến hành một cuộc thăm dò rất lớn, gửi thư cho khoảng 10 triệu người. Họ nhận được 2,4 triệu câu trả lời và dự đoán với độ tin cậy cao rằng Landon sẽ nhận được 57% số phiếu. Thay vào đó, Roosevelt đã thắng với 62% số phiếu. Sai sót nằm ở phương pháp lấy mẫu của Literary Digest :

- Đầu tiên, để có được địa chỉ gửi phiếu thăm dò ý kiến, tạp chí Literary Digest đã sử dụng danh bạ điện thoại, danh sách người đăng ký tạp chí, danh sách thành viên câu lạc bộ, và những nguồn tương tự. Tất cả các danh sách này đều có xu hướng ưu tiên những người giàu có hơn, những người có nhiều khả năng bỏ phiếu cho đảng Cộng hòa (do đó mới có Landon).
- Thứ hai, chưa đến 25% số người được khảo sát đã trả lời.
Điều này lại một lần nữa tạo ra sai lệch trong việc lấy mẫu, bằng cách có khả năng loại trừ những người không quan tâm nhiều đến chính trị, những người không thích tạp chí Literary Digest, và các nhóm quan trọng khác. Đây là một loại sai lệch lấy mẫu đặc biệt được gọi là sai lệch do không phản hồi.

Đây là một ví dụ khác: giả sử bạn muốn xây dựng một hệ thống để nhận dạng các video nhạc funk. Một cách để xây dựng tập dữ liệu huấn luyện là tìm kiếm "nhạc funk" trên YouTube và sử dụng các video kết quả. Nhưng điều này giả định rằng công cụ tìm kiếm của YouTube trả về một tập hợp các video đại diện cho tất cả các video nhạc funk trên YouTube. Trên thực tế, kết quả tìm kiếm có thể bị thiên vị về các nghệ sĩ nổi tiếng (và nếu bạn sống ở Brazil, bạn sẽ nhận được rất nhiều video "funk carioca", nghe không giống James Brown chút nào). Mặt khác, làm thế nào khác bạn có thể có được một tập dữ liệu huấn luyện lớn?

Dữ liệu chất lượng kém

Rõ ràng, nếu dữ liệu huấn luyện của bạn chứa đầy lỗi, giá trị ngoại lệ và nhiễu (ví dụ: do chất lượng đo lường kém), hệ thống sẽ khó phát hiện ra các mẫu tiềm ẩn hơn, do đó hệ thống của bạn sẽ ít có khả năng hoạt động tốt hơn.

Việc dành thời gian để làm sạch dữ liệu huấn luyện thường rất đáng giá.

Thực tế là, hầu hết các nhà khoa học dữ liệu dành một phần đáng kể thời gian của họ để làm việc đó. Sau đây là một vài ví dụ về những trường hợp bạn cần làm sạch dữ liệu huấn luyện:

- Nếu một số trường hợp rõ ràng là ngoại lệ, bạn có thể loại bỏ chúng hoặc thử sửa lỗi thủ công.
- Nếu một số trường hợp thiếu một vài đặc điểm (ví dụ: 5% khách hàng của bạn không chỉ định tuổi), bạn phải quyết định xem mình muốn bỏ qua thuộc tính này hoàn toàn, bỏ qua các trường hợp đó, điền vào các giá trị bị thiếu (ví dụ: bằng tuổi trung vị), hoặc huấn luyện một mô hình có đặc điểm đó và một mô hình không có đặc điểm đó.

Các tính năng không liên quan

Như người ta vẫn nói: "Đầu vào rác, đầu ra rác". Hệ thống của bạn chỉ có khả năng học hỏi nếu dữ liệu huấn luyện chứa đủ các đặc trưng liên quan và không quá nhiều đặc trưng không liên quan. Một phần quan trọng quyết định sự thành công của một dự án Học máy là tìm ra một tập hợp các đặc trưng tốt để huấn luyện. Quá trình này, được gọi là kỹ thuật đặc trưng, bao gồm các bước sau:

- Lựa chọn đặc trưng (chọn ra những đặc trưng hữu ích nhất để huấn luyện trong số các đặc trưng hiện có)
- Trích xuất đặc trưng (kết hợp các đặc trưng hiện có để tạo ra một đặc trưng hữu ích hơn – như chúng ta đã thấy trước đó, các thuật toán giảm chiều có thể giúp ích)
- Tạo ra các tính năng mới bằng cách thu thập dữ liệu mới.

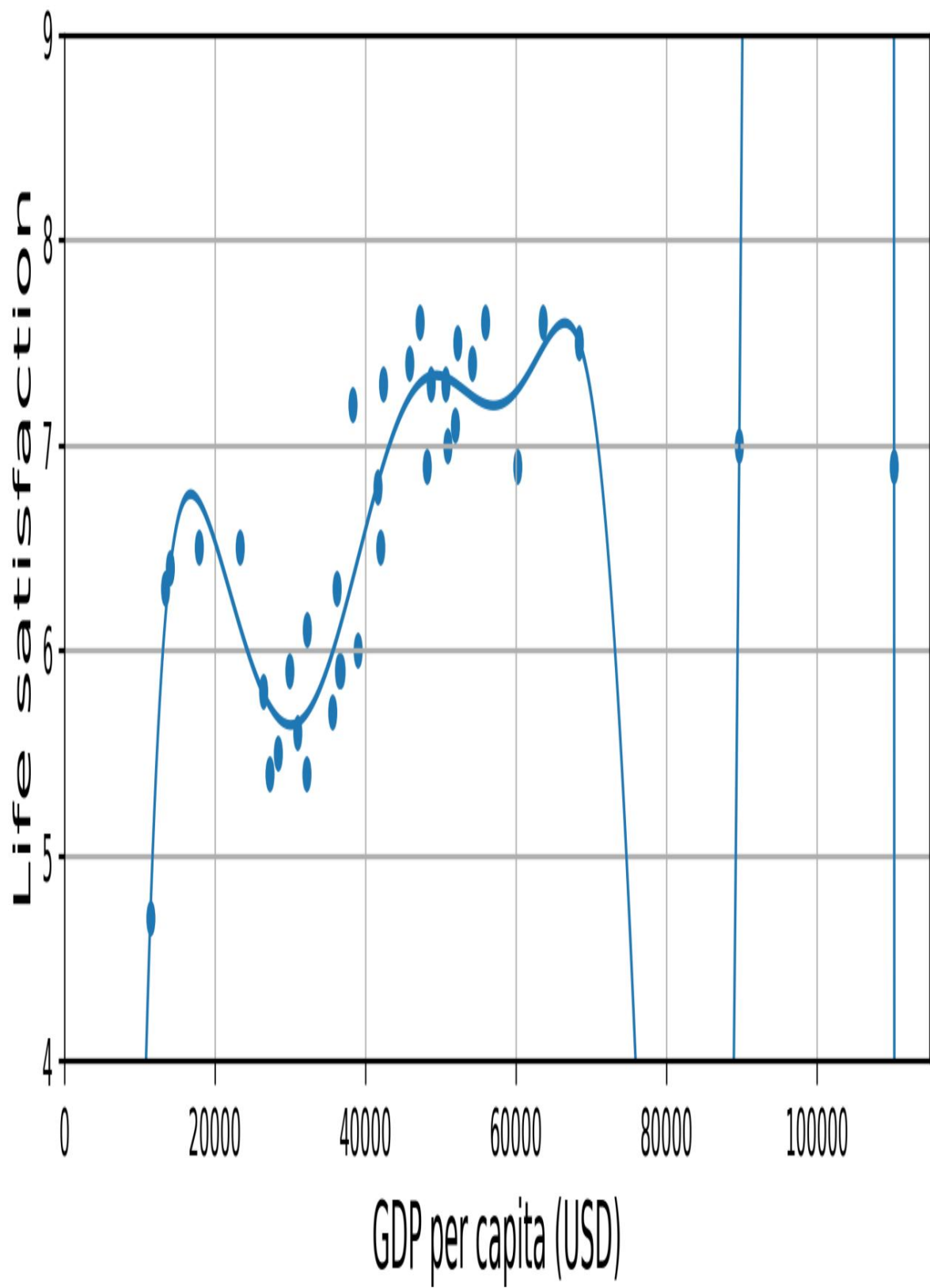
Sau khi đã xem xét nhiều ví dụ về dữ liệu không chính xác, chúng ta hãy cùng xem xét một vài ví dụ về thuật toán không chính xác.

Hiện tượng quá khớp dữ liệu huấn luyện

Giả sử bạn đang đến thăm một quốc gia nước ngoài và tài xế taxi chặt chém bạn. Bạn có thể muốn nói rằng tất cả tài xế taxi ở nước đó đều là kẻ trộm.

Khái quát hóa quá mức là điều mà con người chúng ta thường xuyên mắc phải, và thật không may, máy móc cũng có thể rơi vào cái bẫy tương tự nếu chúng ta không cẩn thận. Trong Học máy, điều này được gọi là hiện tượng quá khớp (overfitting): có nghĩa là mô hình hoạt động tốt trên dữ liệu huấn luyện, nhưng khả năng khái quát hóa lại không tốt.

Hình 1-23 minh họa một ví dụ về mô hình độ hài lòng cuộc sống đa thức bậc cao bị quá khớp dữ liệu huấn luyện. Mặc dù nó hoạt động tốt hơn nhiều trên dữ liệu huấn luyện so với mô hình tuyến tính đơn giản, nhưng liệu bạn có thực sự tin tưởng vào các dự đoán của nó?



Hình 1-23. Hiện tượng quá khớp dữ liệu huấn luyện

Các mô hình phức tạp như mạng nơ-ron sâu có thể phát hiện các mẫu tinh tế trong dữ liệu, nhưng nếu tập dữ liệu huấn luyện bị nhiễu hoặc quá nhỏ (như trong ví dụ về taxi) dẫn đến nhiễu lấy mẫu, thì mô hình có khả năng phát hiện các mẫu trong chính nhiễu đó. Rõ ràng là các mẫu này sẽ không thể khái quát hóa cho các trường hợp mới. Ví dụ, giả sử bạn cung cấp cho mô hình về sự hài lòng trong cuộc sống của mình nhiều thuộc tính hơn, bao gồm cả những thuộc tính không cung cấp thông tin như tên quốc gia. Trong trường hợp đó, một mô hình phức tạp có thể phát hiện các mẫu như việc tất cả các quốc gia trong dữ liệu huấn luyện có chữ "w" trong tên đều có mức độ hài lòng trong cuộc sống lớn hơn 7: New Zealand (7.3), Na Uy (7.6), Thụy Điển (7.3) và Thụy Sĩ (7.5). Bạn tự tin đến mức nào rằng quy tắc hài lòng với chữ "w" có thể khái quát hóa cho Rwanda hoặc Zimbabwe? Rõ ràng mẫu này xuất hiện trong dữ liệu huấn luyện hoàn toàn do ngẫu nhiên, nhưng mô hình không có cách nào để biết liệu một mẫu là có thật hay chỉ đơn giản là kết quả của nhiễu trong dữ liệu.

CẢNH BÁO

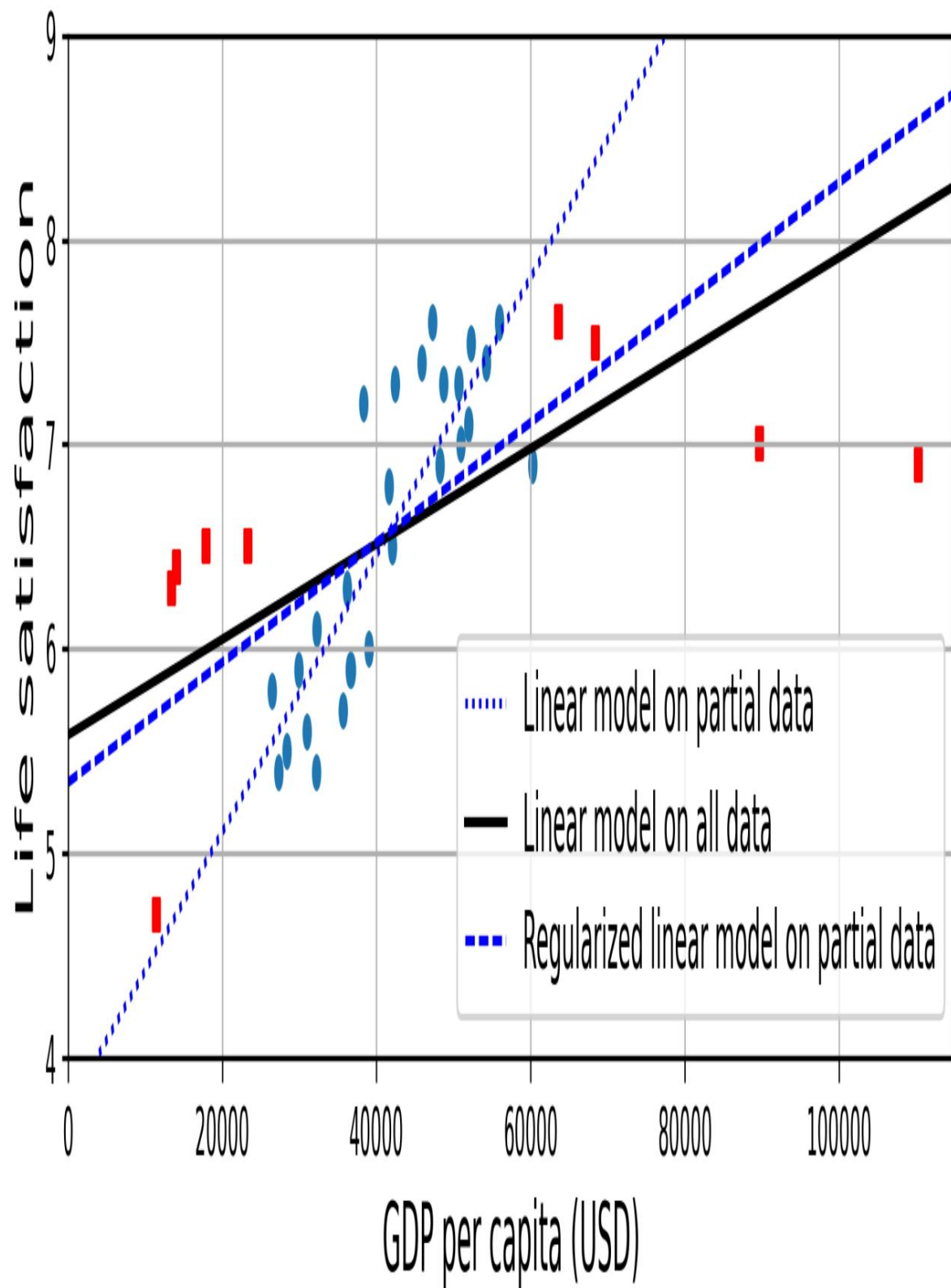
Hiện tượng quá khớp (overfitting) xảy ra khi mô hình quá phức tạp so với lượng dữ liệu huấn luyện và độ nhiễu của nó. Dưới đây là một số giải pháp khả thi:

- Đơn giản hóa mô hình bằng cách chọn mô hình có ít tham số hơn (ví dụ: mô hình tuyến tính thay vì mô hình đa thức bậc cao), bằng cách giảm số lượng thuộc tính trong dữ liệu huấn luyện hoặc bằng cách ràng buộc mô hình.
- Thu thập thêm dữ liệu huấn luyện.
- Giảm nhiễu trong dữ liệu huấn luyện (ví dụ: sửa lỗi dữ liệu và loại bỏ các giá trị ngoại lệ).

Việc ràng buộc một mô hình để làm cho nó đơn giản hơn và giảm nguy cơ quá khớp được gọi là chuẩn hóa (regularization). Ví dụ, mô hình tuyến tính mà chúng ta đã định nghĩa trước đó có hai tham số, θ_0 và θ_1 . Điều này mang lại cho thuật toán học hai mức độ. cho phép nó tự do điều chỉnh mô hình cho phù hợp với dữ liệu huấn luyện: nó có thể điều chỉnh cả chiều cao (θ_0) và độ dốc (θ_1) của đường thẳng. Nếu chúng ta buộc $\theta_1 = 0$, thuật toán sẽ chỉ có một bậc tự do và sẽ gặp khó khăn hơn nhiều trong việc khớp dữ liệu một cách chính xác: tất cả những gì nó có thể làm là di chuyển đường thẳng lên hoặc xuống để càng gần các trường hợp huấn luyện càng tốt, vì vậy nó sẽ nằm xung quanh giá trị trung bình. Một mô hình thực sự rất đơn giản! Nếu chúng ta cho phép thuật toán sửa đổi θ_1 nhưng chúng ta

Nếu buộc nó phải giữ kích thước nhỏ, thuật toán học máy sẽ có số bậc tự do nằm giữa một và hai. Nó sẽ tạo ra một mô hình đơn giản hơn mô hình có hai bậc tự do, nhưng phức tạp hơn mô hình chỉ có một bậc tự do. Bạn cần tìm sự cân bằng phù hợp giữa việc khớp hoàn hảo với dữ liệu huấn luyện và giữ cho mô hình đủ đơn giản để đảm bảo khả năng khái quát hóa tốt.

Hình 1-24 thể hiện ba mô hình. Đường chấm chấm biểu thị mô hình gốc được huấn luyện trên các quốc gia được biểu diễn bằng hình tròn (không bao gồm các quốc gia được biểu diễn bằng hình vuông), đường liền nét là mô hình thứ hai của chúng tôi được huấn luyện với tất cả các quốc gia (hình tròn và hình vuông), và đường nét đứt là mô hình được huấn luyện với cùng dữ liệu như mô hình đầu tiên nhưng có thêm ràng buộc điều chỉnh. Bạn có thể thấy rằng việc điều chỉnh đã buộc mô hình phải có độ dốc nhỏ hơn: mô hình này không khớp với dữ liệu huấn luyện (hình tròn) tốt như mô hình đầu tiên, nhưng nó thực sự khái quát hóa tốt hơn đối với các ví dụ mới mà nó chưa thấy trong quá trình huấn luyện (hình vuông).



Hình 1-24. Chuẩn hóa giúp giảm nguy cơ quá khớp.

Lượng điều chỉnh (regularization) cần áp dụng trong quá trình học có thể được kiểm soát bởi một siêu tham số (hyperparameter). Siêu tham số là một tham số của thuật toán học (không phải của mô hình). Do đó, nó không bị ảnh hưởng bởi chính thuật toán học; nó phải được thiết lập trước khi huấn luyện và giữ nguyên trong suốt quá trình huấn luyện. Nếu bạn đặt siêu tham số điều chỉnh ở một giá trị rất lớn, bạn sẽ nhận được một mô hình gần như phẳng (độ dốc gần bằng không); thuật toán học gần như chắc chắn sẽ không bị quá khớp dữ liệu huấn luyện, nhưng nó sẽ ít có khả năng tìm ra một giải pháp tốt. Điều chỉnh siêu tham số là một phần quan trọng trong việc xây dựng hệ thống Học máy (bạn sẽ thấy một ví dụ chi tiết trong chương tiếp theo).

Hiện tượng thiếu khớp dữ liệu huấn luyện

Như bạn có thể đoán, hiện tượng thiếu khớp (underfitting) là ngược lại với hiện tượng quá khớp (overfitting): nó xảy ra khi mô hình của bạn quá đơn giản để học được cấu trúc cơ bản của dữ liệu. Ví dụ, một mô hình tuyến tính về sự hài lòng trong cuộc sống dễ bị hiện tượng "không phù hợp"; thực tế phức tạp hơn nhiều so với mô hình, vì vậy các dự đoán của nó chắc chắn sẽ không chính xác, ngay cả trên các ví dụ huấn luyện.

Dưới đây là các phương án chính để khắc phục sự cố này:

- Hãy chọn một mô hình mạnh mẽ hơn, với nhiều thông số hơn.
- Cung cấp các đặc trưng tốt hơn cho thuật toán học máy (kỹ thuật đặc trưng).
- Giảm bớt các ràng buộc đối với mô hình (ví dụ: giảm tham số siêu điều chỉnh).

Lùi lại

Đến giờ bạn đã biết khá nhiều về Học máy. Tuy nhiên, chúng ta đã đi qua rất nhiều khái niệm đến nỗi bạn có thể cảm thấy hơi bối rối, vậy nên hãy cùng nhìn lại bức tranh tổng thể nhé:

- Học máy là việc giúp máy móc thực hiện một nhiệm vụ nào đó tốt hơn bằng cách học hỏi từ dữ liệu, thay vì phải lập trình các quy tắc một cách rõ ràng.

- Có rất nhiều loại hệ thống học máy khác nhau: có giám sát hoặc không giám sát, xử lý theo lô hoặc trực tuyến, dựa trên trường hợp cụ thể hoặc dựa trên mô hình.
- Trong một dự án học máy, bạn thu thập dữ liệu vào một tập huấn luyện, và đưa tập huấn luyện đó cho một thuật toán học. Nếu thuật toán dựa trên mô hình, nó sẽ điều chỉnh một số tham số để phù hợp mô hình với tập huấn luyện (tức là để đưa ra dự đoán tốt trên chính tập huấn luyện đó), và sau đó hy vọng nó cũng sẽ có thể đưa ra dự đoán tốt trên các trường hợp mới. Nếu thuật toán dựa trên trường hợp cụ thể, nó chỉ học thuộc lòng các ví dụ và khái quát hóa cho các trường hợp mới bằng cách sử dụng một thước đo độ tương đồng để so sánh chúng với các trường hợp đã học.
- Hệ thống sẽ không hoạt động tốt nếu tập dữ liệu huấn luyện quá nhỏ, hoặc nếu dữ liệu không mang tính đại diện, bị nhiễu, hoặc bị lẫn các đặc trưng không liên quan (đầu vào rác, đầu ra rác). Cuối cùng, mô hình của bạn cần phải không quá đơn giản (trong trường hợp đó sẽ bị thiếu dữ liệu) cũng không quá phức tạp (trong trường hợp đó sẽ bị quá khớp).

Chỉ còn một chủ đề quan trọng cuối cùng cần đề cập: sau khi đã huấn luyện mô hình, bạn không nên chỉ "hy vọng" nó sẽ khái quát hóa tốt cho các trường hợp mới. Bạn cần đánh giá và tinh chỉnh nó nếu cần thiết. Hãy cùng xem cách thực hiện điều đó.

Kiểm thử và xác nhận

Cách duy nhất để biết một mô hình sẽ tổng quát hóa tốt như thế nào đối với các trường hợp mới là thực sự thử nghiệm nó trên các trường hợp mới. Một cách để làm điều đó là đưa mô hình của bạn vào sản xuất và theo dõi hiệu suất của nó. Cách này hoạt động tốt, nhưng nếu mô hình của bạn quá tệ, người dùng sẽ phàn nàn - đó không phải là ý tưởng hay.

Một lựa chọn tốt hơn là chia dữ liệu của bạn thành hai tập: tập huấn luyện và tập kiểm tra. Như tên gọi đã thể hiện, bạn huấn luyện mô hình bằng tập huấn luyện và kiểm tra nó bằng tập kiểm tra. Tỷ lệ lỗi trên các trường hợp mới được gọi là lỗi tổng quát hóa (hoặc lỗi ngoài mẫu), và bằng cách đánh giá mô hình của bạn trên tập kiểm tra, bạn sẽ nhận được ước tính về lỗi này. Giá trị này cho bạn biết mô hình của bạn sẽ hoạt động tốt như thế nào trên các trường hợp mà nó chưa từng thấy trước đây.

Nếu lỗi huấn luyện thấp (tức là mô hình của bạn mắc ít lỗi trên tập dữ liệu huấn luyện) nhưng lỗi tổng quát hóa cao, điều đó có nghĩa là mô hình của bạn đang bị quá khớp với dữ liệu huấn luyện.

MẸO

Thông thường, người ta sử dụng 80% dữ liệu để huấn luyện và giữ lại 20% để kiểm tra. Tuy nhiên, điều này phụ thuộc vào kích thước của tập dữ liệu: nếu nó chứa 10 triệu trường hợp, thì việc giữ lại 1% có nghĩa là tập dữ liệu kiểm tra của bạn sẽ chứa 100.000 trường hợp, có lẽ là quá đủ để ước tính tốt lỗi tổng quát hóa.

Điều chỉnh siêu tham số và lựa chọn mô hình

Việc đánh giá một mô hình khá đơn giản: chỉ cần sử dụng tập dữ liệu kiểm thử. Nhưng giả sử bạn đang phân vân giữa hai loại mô hình (ví dụ, mô hình tuyến tính và mô hình đa thức): làm thế nào bạn có thể quyết định? Một lựa chọn là huấn luyện cả hai và so sánh khả năng khái quát hóa của chúng bằng cách sử dụng tập dữ liệu kiểm thử.

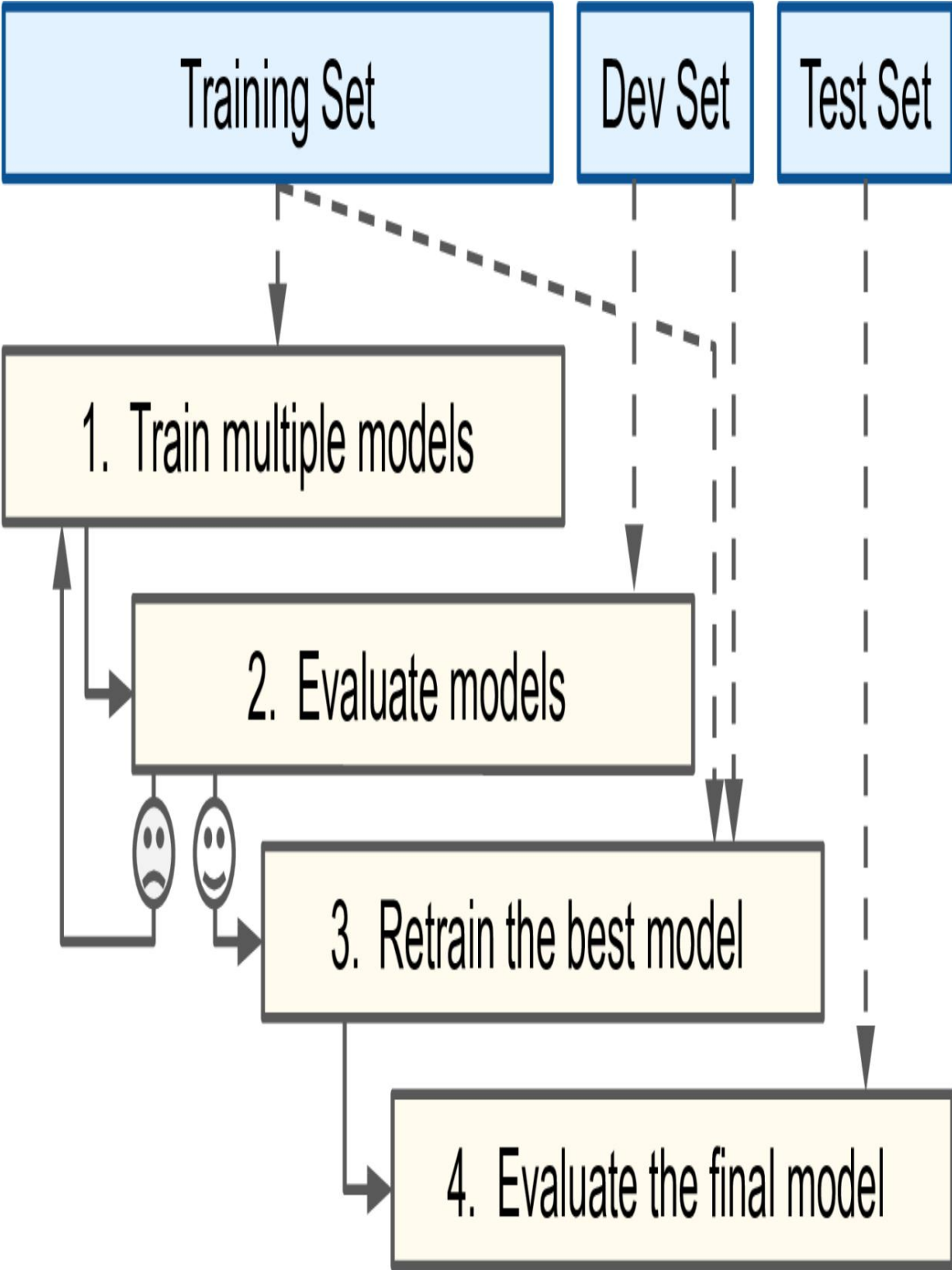
Giả sử mô hình tuyến tính có khả năng tổng quát hóa tốt hơn, nhưng bạn muốn áp dụng một số phương pháp điều chỉnh (regularization) để tránh hiện tượng quá khớp (overfitting). Câu hỏi đặt ra là, làm thế nào để chọn giá trị của siêu tham số điều chỉnh? Một lựa chọn là huấn luyện 100 mô hình khác nhau sử dụng 100 giá trị khác nhau cho siêu tham số này. Giả sử bạn tìm thấy giá trị siêu tham số tốt nhất tạo ra một mô hình có lỗi tổng quát hóa thấp nhất—ví dụ, chỉ 5% lỗi.

Bạn đưa mô hình này vào sản xuất, nhưng không may là nó không hoạt động tốt như mong đợi và tạo ra 15% lỗi. Chuyện gì vừa xảy ra vậy?

Vấn đề là bạn đã đo lỗi tổng quát hóa nhiều lần trên tập dữ liệu thử nghiệm, và bạn đã điều chỉnh mô hình và các siêu tham số để tạo ra mô hình tốt nhất cho tập dữ liệu cụ thể đó. Điều này có nghĩa là mô hình khó có thể hoạt động tốt trên dữ liệu mới.

Một giải pháp phổ biến cho vấn đề này được gọi là xác thực bằng cách giữ lại (holdout validation) (Hình 1-25): bạn chỉ cần giữ lại một phần của tập dữ liệu huấn luyện để đánh giá một số mô hình ứng cử viên và chọn mô hình tốt nhất. Tập dữ liệu được giữ lại này được gọi là tập dữ liệu xác thực (hoặc tập dữ liệu phát triển, hoặc tập dữ liệu dev). Cụ thể hơn, bạn

Huấn luyện nhiều mô hình với các siêu tham số khác nhau trên tập dữ liệu huấn luyện rút gọn (tức là toàn bộ tập dữ liệu huấn luyện trừ đi tập dữ liệu kiểm định), và bạn chọn mô hình hoạt động tốt nhất trên tập dữ liệu kiểm định. Sau quá trình kiểm định giữ lại này, bạn huấn luyện mô hình tốt nhất trên toàn bộ tập dữ liệu huấn luyện (bao gồm cả tập dữ liệu kiểm định), và điều này sẽ cho bạn mô hình cuối cùng. Cuối cùng, bạn đánh giá mô hình cuối cùng này trên tập dữ liệu kiểm thử để ước tính sai số tổng quát.



Hình 1-25. Lựa chọn mô hình bằng phương pháp kiểm định độc lập.

Giải pháp này thường hoạt động khá tốt. Tuy nhiên, nếu tập dữ liệu kiểm định quá nhỏ, thì việc đánh giá mô hình sẽ không chính xác: bạn có thể chọn nhầm một mô hình không tối ưu. Ngược lại, nếu tập dữ liệu kiểm định quá lớn, thì tập dữ liệu huấn luyện còn lại sẽ nhỏ hơn nhiều so với toàn bộ tập dữ liệu huấn luyện. Tại sao điều này lại không tốt? Bởi vì mô hình cuối cùng sẽ được huấn luyện trên toàn bộ tập dữ liệu huấn luyện, nên việc so sánh các mô hình ứng cử viên được huấn luyện trên một tập dữ liệu huấn luyện nhỏ hơn nhiều là không lý tưởng. Nó giống như việc chọn vận động viên chạy nước rút nhanh nhất để tham gia một cuộc chạy marathon. Một cách để giải quyết vấn đề này là thực hiện kiểm định chéo lặp lại, sử dụng nhiều tập dữ liệu kiểm định nhỏ. Mỗi mô hình được đánh giá một lần trên mỗi tập dữ liệu kiểm định sau khi nó được huấn luyện trên phần dữ liệu còn lại. Bằng cách tính trung bình tất cả các đánh giá của một mô hình, bạn sẽ có được một thước đo chính xác hơn nhiều về hiệu suất của nó. Tuy nhiên, có một nhược điểm: thời gian huấn luyện được nhân lên bởi số lượng tập dữ liệu kiểm định.

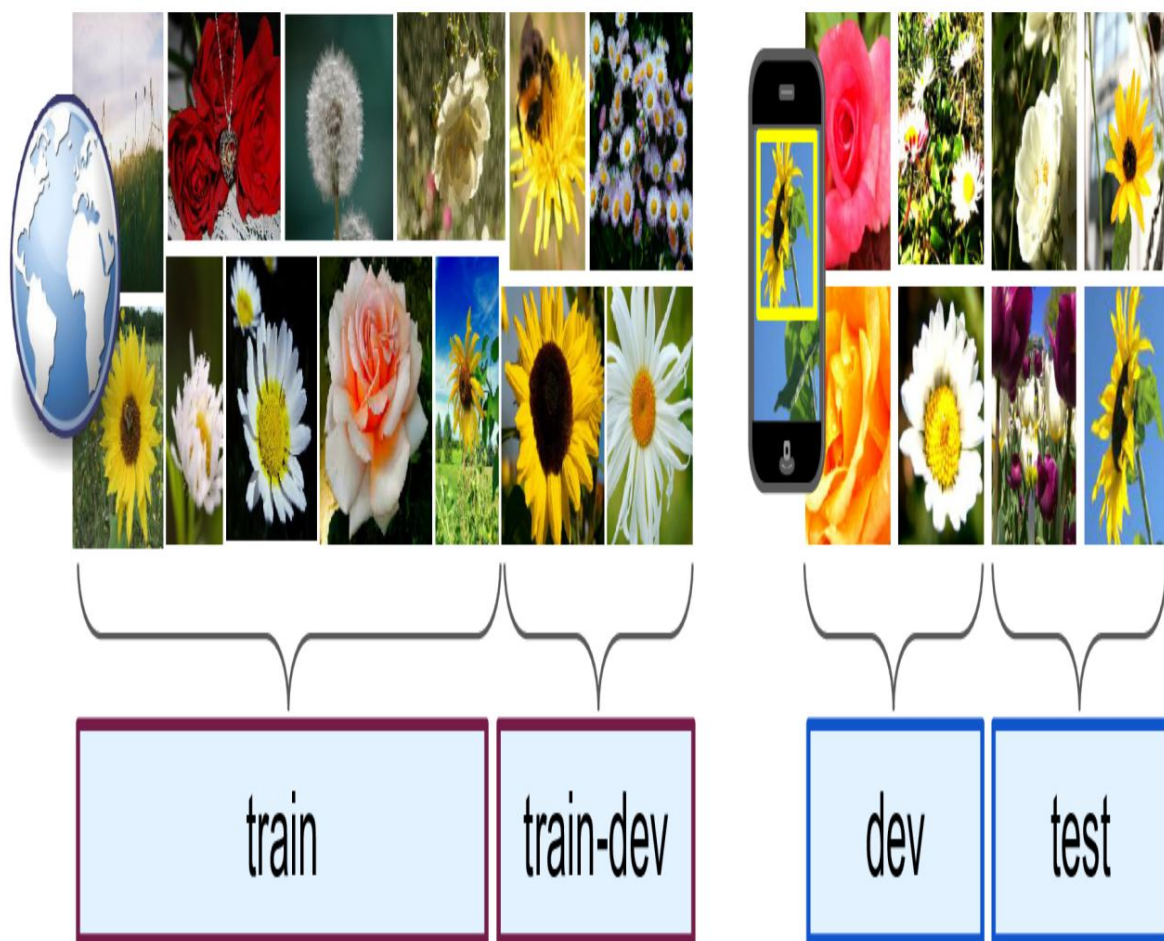
Dữ liệu không khớp

Trong một số trường hợp, việc thu thập một lượng lớn dữ liệu để huấn luyện rất dễ dàng, nhưng dữ liệu này có thể không hoàn toàn đại diện cho dữ liệu sẽ được sử dụng trong môi trường sản xuất. Ví dụ, giả sử bạn muốn tạo một ứng dụng di động để chụp ảnh hoa và tự động xác định loài của chúng. Bạn có thể dễ dàng tải xuống hàng triệu bức ảnh hoa trên web, nhưng chúng sẽ không hoàn toàn đại diện cho những bức ảnh thực tế được chụp bằng ứng dụng trên thiết bị di động. Có thể bạn chỉ có 1.000 bức ảnh đại diện (tức là những bức ảnh thực sự được chụp bằng ứng dụng).

Trong trường hợp này, quy tắc quan trọng nhất cần nhớ là cả tập dữ liệu xác thực và tập dữ liệu kiểm thử phải càng đại diện cho dữ liệu bạn dự định sử dụng trong môi trường sản xuất càng tốt, vì vậy chúng chỉ nên bao gồm các hình ảnh mang tính đại diện: bạn có thể xáo trộn chúng và chia một nửa vào tập dữ liệu xác thực và một nửa vào tập dữ liệu kiểm thử (đảm bảo không có hình ảnh trùng lặp hoặc gần trùng lặp nào xuất hiện trong cả hai tập dữ liệu). Sau khi huấn luyện mô hình của bạn trên các hình ảnh từ web, nếu bạn nhận thấy hiệu suất của mô hình trên tập dữ liệu xác thực không đạt yêu cầu, bạn sẽ không biết liệu điều này là do mô hình của bạn đã quá khớp với tập dữ liệu huấn luyện, hay chỉ đơn giản là do sự không khớp giữa các hình ảnh từ web và hình ảnh từ ứng dụng di động.

Một giải pháp là giữ lại một số hình ảnh huấn luyện (lấy từ web) trong một tập dữ liệu khác mà Andrew Ng gọi là tập dữ liệu huấn luyện-phát triển (Hình 1-26).

Sau khi mô hình được huấn luyện (trên tập dữ liệu huấn luyện, không phải trên tập dữ liệu huấn luyện-phát triển), bạn có thể đánh giá nó trên tập dữ liệu huấn luyện-phát triển. Nếu mô hình hoạt động kém, thì có thể nó đã bị quá khớp với tập dữ liệu huấn luyện, vì vậy bạn nên thử đơn giản hóa hoặc điều chỉnh mô hình, thu thập thêm dữ liệu huấn luyện và làm sạch dữ liệu huấn luyện. Nhưng nếu mô hình hoạt động tốt trên tập dữ liệu huấn luyện-phát triển, thì bạn có thể đánh giá mô hình trên tập dữ liệu phát triển. Nếu nó hoạt động kém, thì vấn đề có thể đến từ sự không khớp dữ liệu. Bạn có thể thử giải quyết vấn đề này bằng cách tiền xử lý các hình ảnh trên web để chúng trông giống hơn với những bức ảnh sẽ được chụp bởi ứng dụng di động, và sau đó huấn luyện lại mô hình. Khi bạn có một mô hình hoạt động tốt trên cả tập dữ liệu huấn luyện-phát triển và tập dữ liệu phát triển, bạn có thể đánh giá nó lần cuối trên tập dữ liệu kiểm tra để biết nó có khả năng hoạt động tốt như thế nào trong môi trường sản xuất.



Hình 1-26. Khi dữ liệu thực tế khan hiếm (bên phải), bạn có thể sử dụng dữ liệu tương tự dồi dào (bên trái) để huấn luyện và giữ lại một phần trong tập huấn luyện-phát triển để đánh giá hiện tượng quá khớp. Dữ liệu thực tế sau đó được sử dụng để đánh giá sự không khớp dữ liệu (tập phát triển) và để đánh giá hiệu suất của mô hình cuối cùng (tập kiểm tra).

ĐẠO LÝ KHÔNG CÓ BỮA TRƯA MIỄN PHÍ

Mô hình là một biểu diễn đơn giản hóa của dữ liệu. Việc đơn giản hóa nhằm mục đích loại bỏ các chi tiết thừa không có khả năng khái quát hóa cho các trường hợp mới.

Khi bạn chọn một loại mô hình cụ thể, bạn đang ngầm đưa ra các giả định về dữ liệu. Ví dụ, nếu bạn chọn mô hình tuyến tính, bạn đang ngầm giả định rằng dữ liệu về cơ bản là tuyến tính và khoảng cách giữa các trường hợp và đường thẳng chỉ là nhiễu, có thể bỏ qua một cách an toàn.

Trong một [bài báo nổi tiếng năm 1996](#)⁹, David Wolpert đã chứng minh rằng nếu bạn hoàn toàn không đưa ra bất kỳ giả định nào về dữ liệu, thì không có lý do gì để ưu tiên mô hình này hơn mô hình khác. Điều này được gọi là định lý Không Có Bữa Trưa Miễn Phí (No Free Lunch - NFL). Đối với một số tập dữ liệu, mô hình tốt nhất là mô hình tuyến tính, trong khi đối với các tập dữ liệu khác, đó lại là mạng nơ-ron. Không có mô hình nào được đảm bảo trước là hoạt động tốt hơn (do đó có tên của định lý). Cách duy nhất để biết chắc chắn mô hình nào là tốt nhất là đánh giá tất cả các mô hình. Vì điều này là không thể, trên thực tế bạn đưa ra một số giả định hợp lý về dữ liệu và chỉ đánh giá một vài mô hình hợp lý. Ví dụ, đối với các tác vụ đơn giản, bạn có thể đánh giá các mô hình tuyến tính với các mức độ điều chỉnh khác nhau, còn đối với các vấn đề phức tạp, bạn có thể đánh giá các mạng nơ-ron khác nhau.

Bài tập

Trong chương này, chúng ta đã đề cập đến một số khái niệm quan trọng nhất trong Học máy. Trong các chương tiếp theo, chúng ta sẽ đi sâu hơn và viết nhiều mã hơn, nhưng trước khi làm điều đó, hãy đảm bảo bạn có thể trả lời các câu hỏi sau:

1. Bạn định nghĩa Học máy như thế nào?
2. Bạn có thể kể tên bốn loại ứng dụng mà nó thể hiện ưu điểm vượt trội không?
3. Tập dữ liệu huấn luyện có nhãn là gì?

4. Hai nhiệm vụ được giám sát phổ biến nhất là gì?
5. Bạn có thể kể tên bốn nhiệm vụ không cần giám sát phổ biến không?
6. Bạn sẽ sử dụng loại thuật toán nào để cho phép robot đi bộ?
nhiều địa hình chưa được biết đến?
7. Bạn sẽ sử dụng loại thuật toán nào để phân chia khách hàng thành nhiều nhóm khác nhau?
8. Bạn sẽ coi bài toán phát hiện thư rác là bài toán học có giám sát hay bài toán học
không giám sát?
9. Hệ thống học trực tuyến là gì?
10. Học tập ngoài chương trình giảng dạy cốt lõi là gì?
11. Loại thuật toán nào dựa vào thước đo độ tương đồng để đưa ra dự đoán?
12. Sự khác biệt giữa tham số mô hình và siêu tham số mô hình là gì?
13. Các thuật toán dựa trên mô hình tìm kiếm điều gì? Điều quan trọng nhất là gì?
Họ thường sử dụng chiến lược nào để thành công? Họ dự đoán như thế nào?
14. Bạn có thể kể tên bốn thách thức chính trong Học máy không?
15. Nếu mô hình của bạn hoạt động tốt trên dữ liệu huấn luyện nhưng lại khá quát hóa kém
đối với các trường hợp mới, điều gì đang xảy ra? Bạn có thể nêu ra ba giải pháp khả
thi không?
16. Tập dữ liệu kiểm thử là gì và tại sao bạn lại muốn sử dụng nó?
17. Mục đích của tập dữ liệu kiểm định là gì?
18. Tập dữ liệu huấn luyện-phát triển là gì, khi nào bạn cần đến nó và bạn sử dụng nó như thế nào?
19. Điều gì có thể xảy ra sai sót nếu bạn điều chỉnh siêu tham số bằng cách sử dụng tập dữ liệu kiểm thử?

Đáp án cho các bài tập này có sẵn ở cuối tập tài liệu của chương này, tại địa chỉ <https://homl.info/colab3>.

-
1. Một sự thật thú vị: Cái tên nghe có vẻ lạ này là một thuật ngữ thống kê do Francis Galton đưa ra khi ông... Ông đã nghiên cứu thực tế rằng con cái của những người cao thường thấp hơn cha mẹ chúng. Vì những đứa trẻ này thấp hơn, ông gọi hiện tượng này là sự hồi quy về giá trị trung bình. Tên gọi này sau đó được áp dụng cho các phương pháp ông sử dụng để phân tích mối tương quan giữa các biến số.
 2. Hãy chú ý cách các loài động vật được tách biệt khá rõ ràng khỏi các phương tiện giao thông và cách ngựa ở gần hươu nhưng lại xa chim. Hình ảnh được sao chép với sự cho phép từ Richard Socher và cộng sự, "Học tập không cần dữ liệu huấn luyện thông qua chuyển giao đa phương thức", Kỷ yếu Hội nghị quốc tế lần thứ 26 về Hệ thống xử lý thông tin thần kinh 1 (2013): 935-943.
 3. Đó là khi hệ thống hoạt động hoàn hảo. Trên thực tế, nó thường tạo ra một vài nhóm cho mỗi người, và đôi khi nhầm lẫn hai người trông giống nhau, vì vậy bạn có thể cần cung cấp một vài nhãn cho mỗi người và tự tay dọn dẹp một số nhóm.
 4. Theo quy ước, chữ cái Hy Lạp θ (theta) thường được sử dụng để biểu thị các tham số của mô hình.
 5. Không sao nếu bạn chưa hiểu hết mã; tôi sẽ giới thiệu Scikit-Learn ở phần tiếp theo. chương.
 6. Ví dụ, biết nên viết "to," "two," hay "too," tùy thuộc vào ngữ cảnh.
- Hình 7 được sao chép lại với sự cho phép từ Michele Banko và Eric Brill, "Mở rộng quy mô lên các tập dữ liệu rất rất lớn để phân giải mơ hồ ngôn ngữ tự nhiên", Kỷ yếu Hội nghị thường niên lần thứ 39 của Hiệp hội Ngôn ngữ học tính toán (2001): 26-33.
- 8 Peter Norvig và cộng sự, "Hiệu quả phi lý của dữ liệu", IEEE Intelligent Systems 24, số 2 (2009): 8-12.
 - 9 David Wolpert, "Sự thiếu phân biệt tiên nghiệm giữa các thuật toán học tập," Tính toán thần kinh 8, số 7 (1996): 1341-1390.

Chương 2. Dự án học máy từ đầu đến cuối

LƯU Ý DÀNH CHO ĐỘC GIẢ ĐẶT MUA SỚM

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất—nội dung thô và chưa chỉnh sửa của tác giả khi họ viết—nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 2 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Trong chương này, bạn sẽ thực hiện một dự án ví dụ từ đầu đến cuối, đóng vai một nhà khoa học dữ liệu mới được tuyển dụng tại một công ty bất động sản. Dự án ví dụ này là hư cấu; mục tiêu là để minh họa các bước chính của một dự án Học máy, chứ không phải để học bất cứ điều gì về ngành kinh doanh bất động sản. Dưới đây là các bước chính mà chúng ta sẽ cùng nhau thực hiện:

1. Hãy nhìn vào bức tranh toàn cảnh.
2. Thu thập dữ liệu.
3. Khám phá và trực quan hóa dữ liệu để thu được thông tin chi tiết.
4. Chuẩn bị dữ liệu cho các thuật toán Học máy.
5. Chọn một mô hình và huấn luyện nó.
6. Tinh chỉnh mô hình của bạn.

7. Trình bày giải pháp của bạn.

8. Khởi chạy, giám sát và bảo trì hệ thống của bạn.

Làm việc với dữ liệu thực tế

Khi học về Máy học, tốt nhất là nên thực hành với dữ liệu thực tế, chứ không phải các tập dữ liệu nhân tạo. May mắn thay, có hàng ngàn tập dữ liệu mở để lựa chọn, trải rộng trên nhiều lĩnh vực khác nhau. Dưới đây là một vài nơi bạn có thể tìm kiếm dữ liệu:

- Các kho dữ liệu mở phổ biến
 - [OpenML.org](https://openml.org)
 - [Kaggle.com](https://kaggle.com)
 - [PapersWithCode.com](https://paperswithcode.com)
 - Kho lưu trữ học máy UC Irvine
 - Bộ dữ liệu AWS của Amazon
 - Bộ dữ liệu TensorFlow
- Cổng thông tin tổng hợp (liệt kê các kho dữ liệu mở)
 - [DataPortals.org](https://dataportals.org)
 - [OpenDataMonitor.eu](https://opendatamonitor.eu)
- Các trang khác liệt kê nhiều kho dữ liệu mở phổ biến.
 - Danh sách các bộ dữ liệu Học máy của Wikipedia
 - [Quora.com](https://www.quora.com)
 - Subreddit về tập dữ liệu

Trong chương này, chúng ta sẽ sử dụng bộ dữ liệu Giá nhà ở California từ [nguồn dữ liệu].

Kho lưu trữ StatLib (xem [Hình 2-1](#)¹). Tập dữ liệu này dựa trên dữ liệu từ...

Dữ liệu điều tra dân số California năm 1990. Mặc dù không hoàn toàn mới (một ngôi nhà đẹp ở khu vực Vịnh San Francisco vẫn còn khá phải chăng vào thời điểm đó), nhưng nó có nhiều đặc điểm hữu ích cho việc học tập, vì vậy chúng ta sẽ coi như đây là dữ liệu gần đây. Để phục vụ mục đích giảng dạy, tôi đã thêm một thuộc tính phân loại và loại bỏ một vài đặc điểm khác.

